

# Computer Networks

## Chapter 1 - Foundations

Prof. Marino Miculan



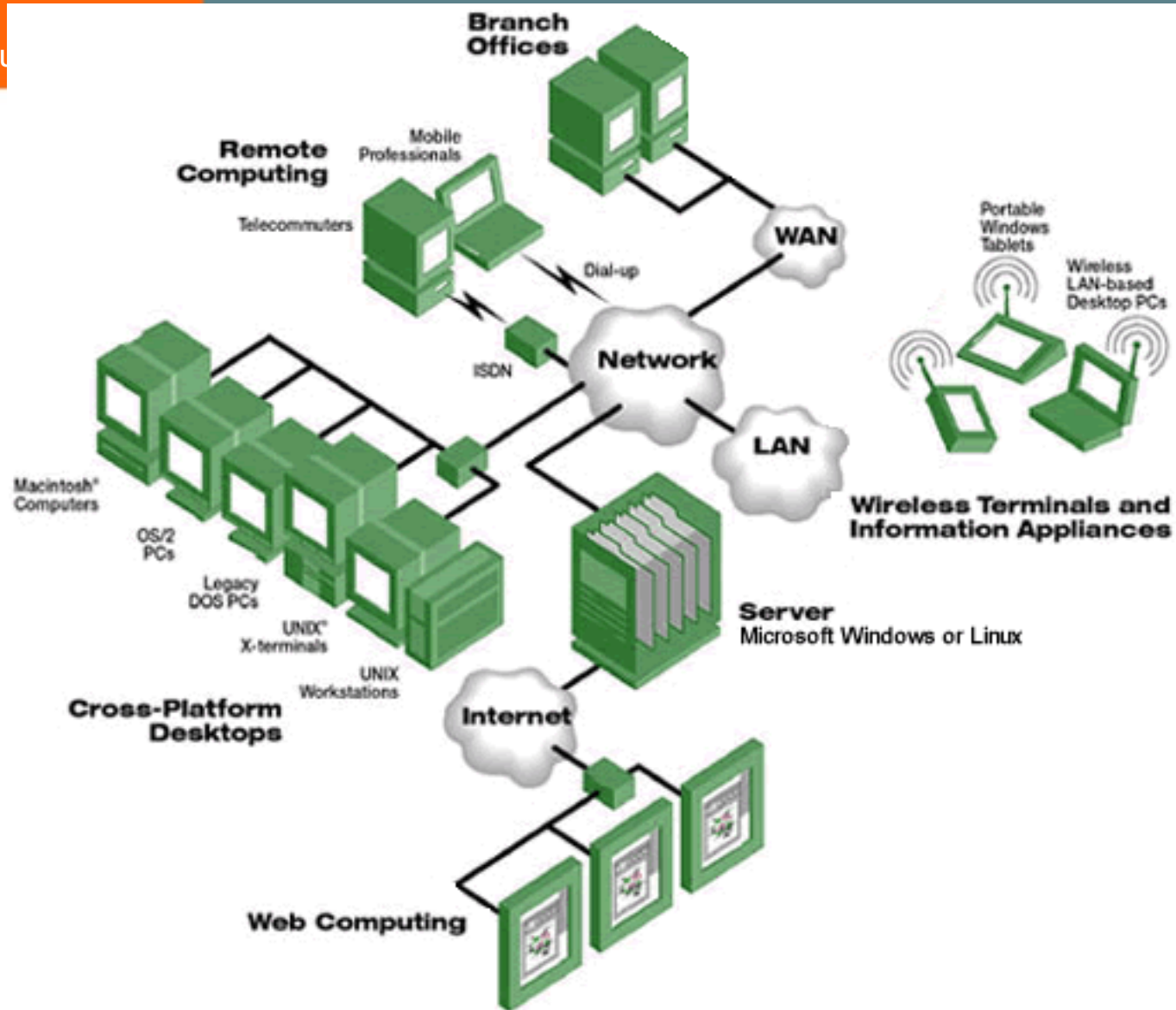
# Prologue

# What?

- Suppose we want to build, or choose, or manage, a computer network
  - Who is going to use this network?
  - Why? which are the applications?
  - Which architecture?
  - What hardware/software?
- What is a **(computer) network**?
  - How is a computer network different from other types of networks?
- What is a **computer network architecture**?

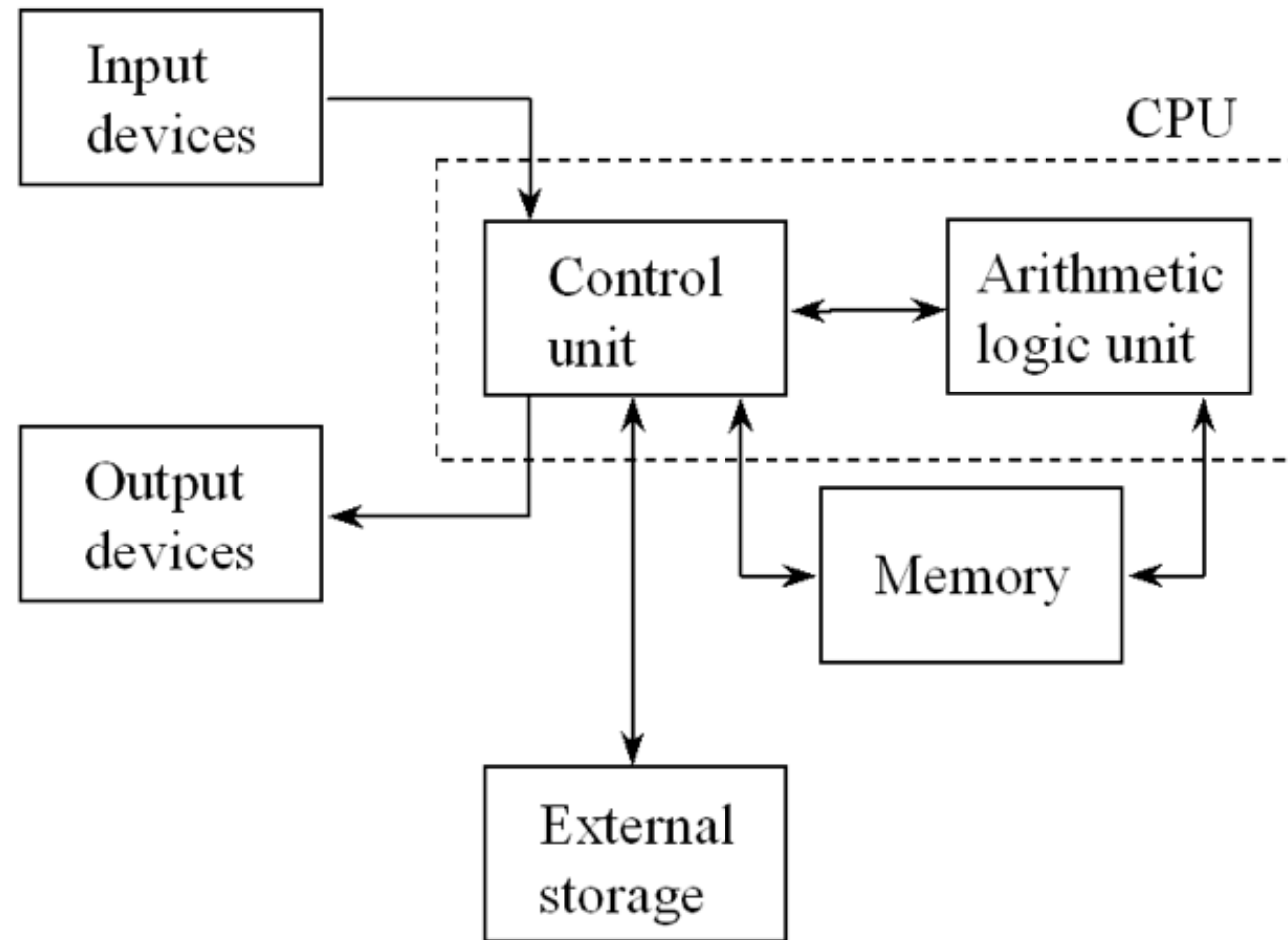


Ceci n'est pas un reseau.



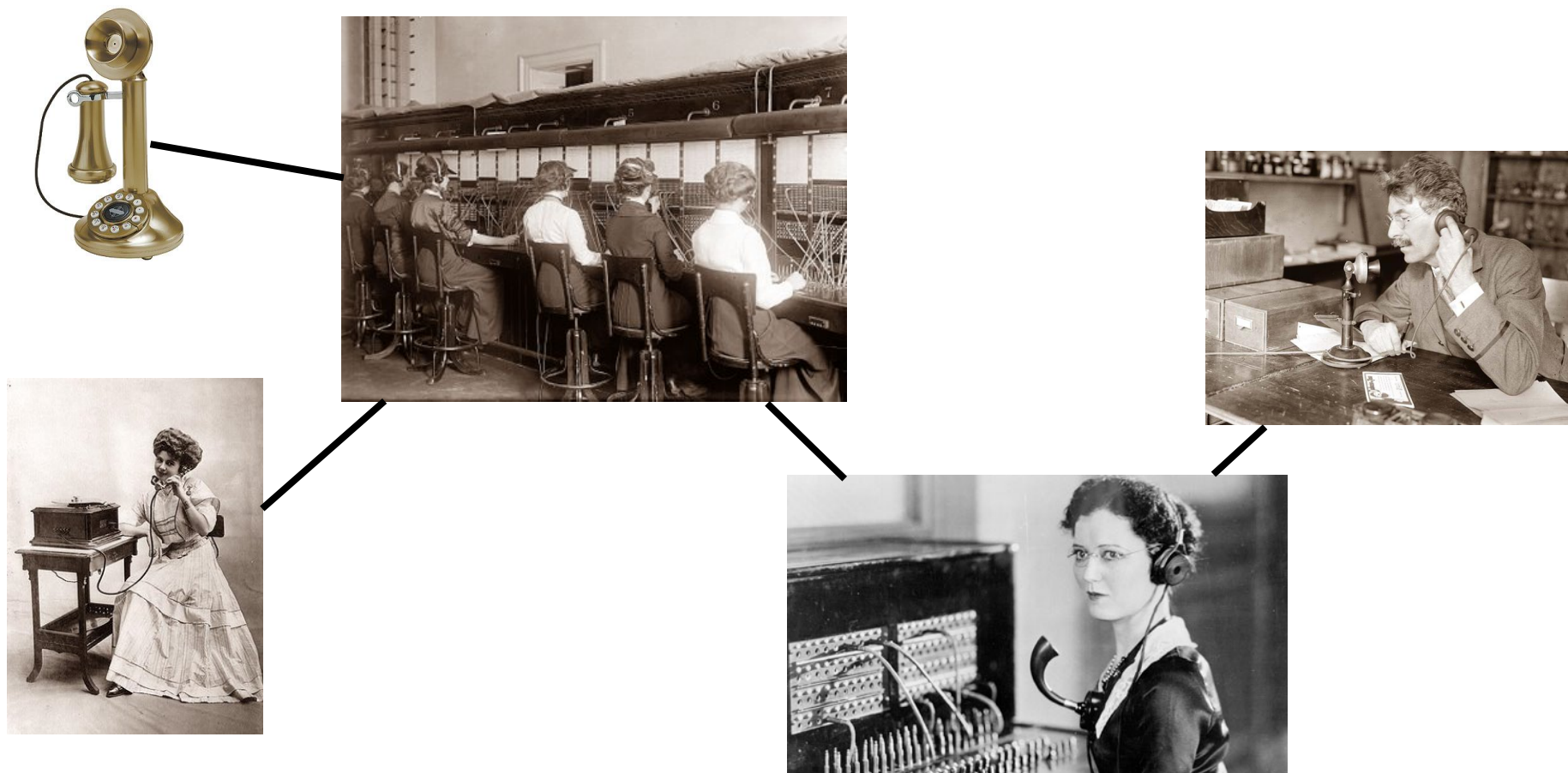
# Ceci est un reseau.

Many different independent nodes, each with its own computing power



# Ceci n'est pas un reseau.

One single “control” node, the others have no computing power



# Ceci est un reseau.

Dynamically established connections, via intermediary specialized nodes and channels. However, designed for a very specific application.



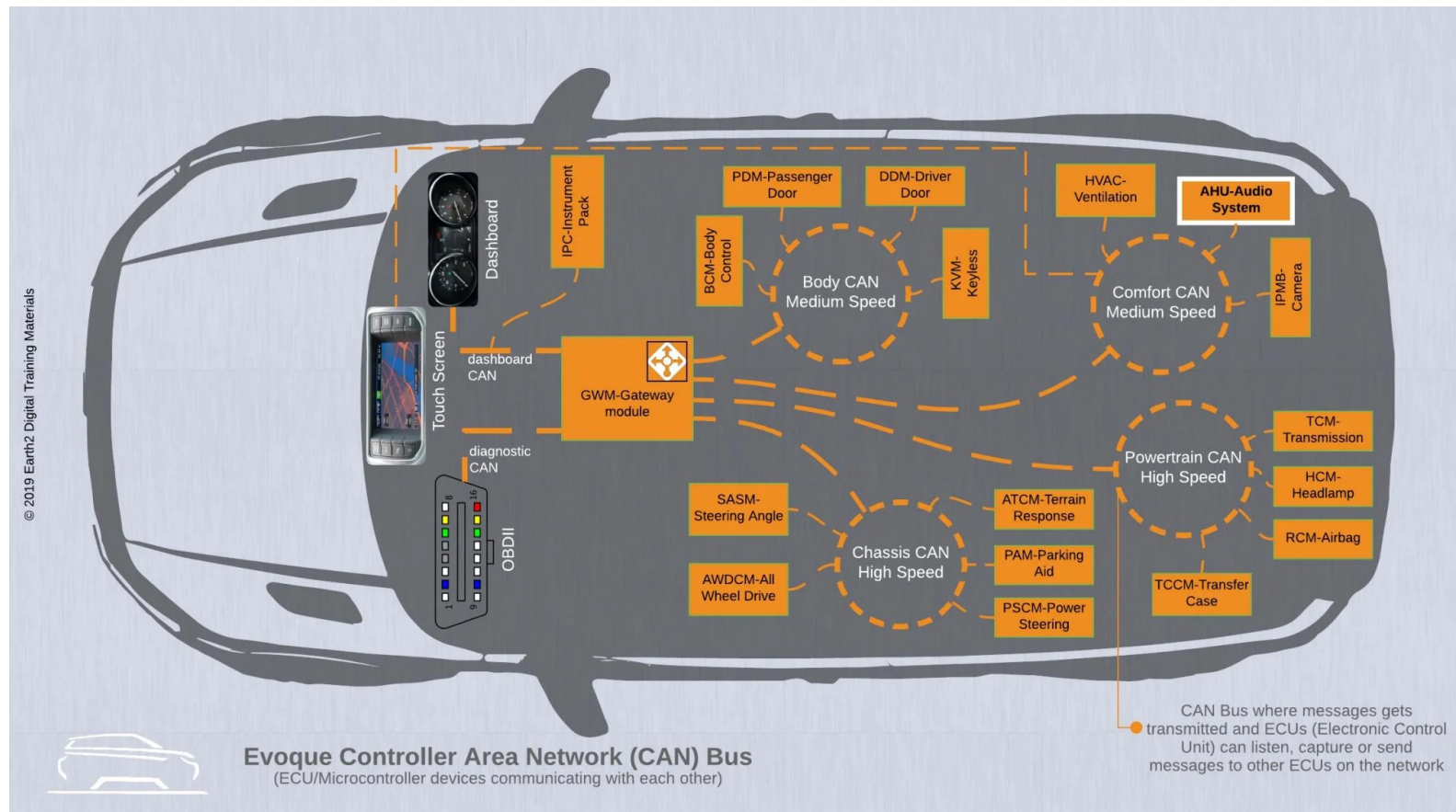
# Ceci est un reseau.

Dynamically created links, “ad hoc” network



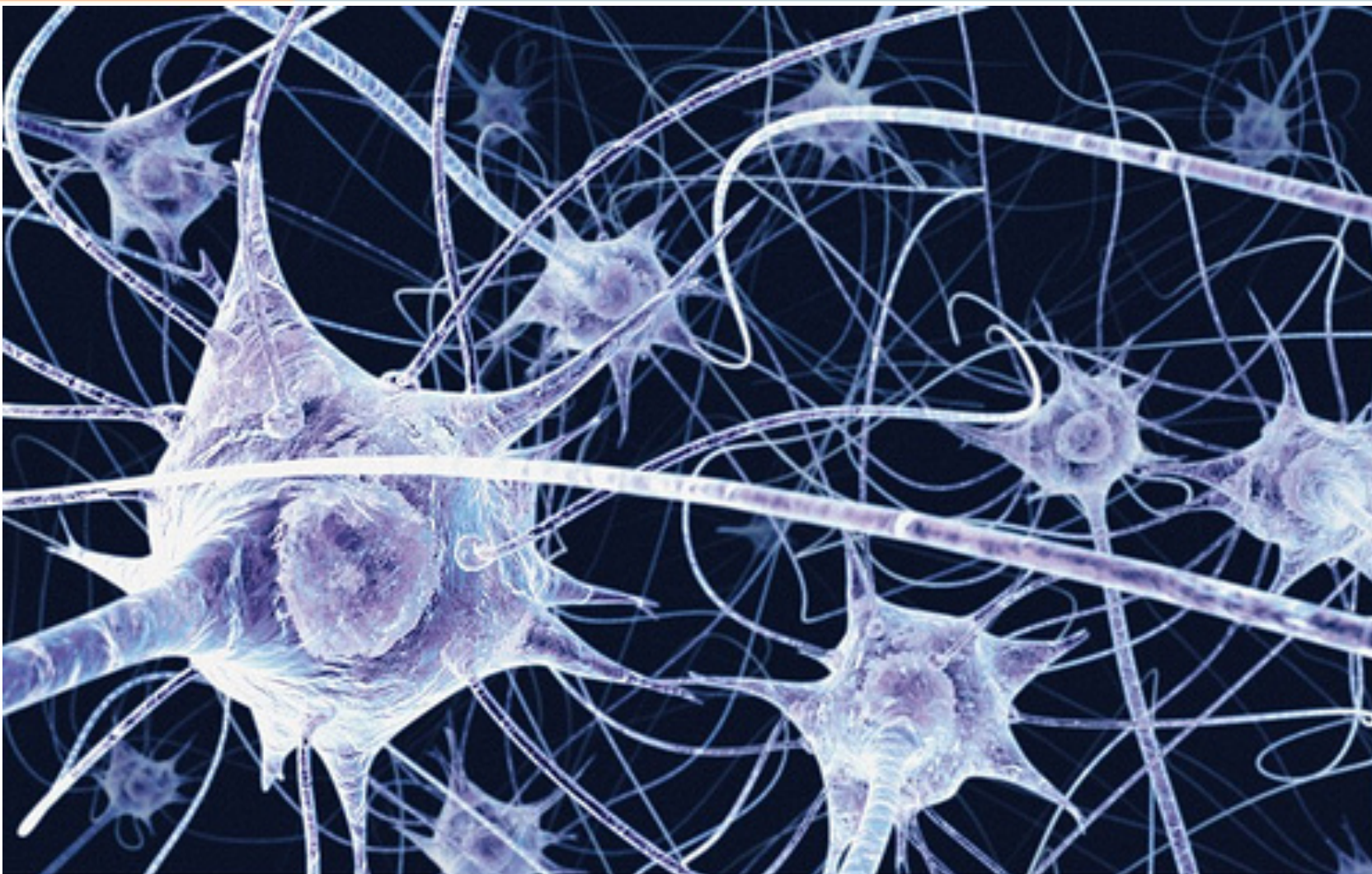
## Ceci est un reseau.

Many different nodes join forces to reach a common goal



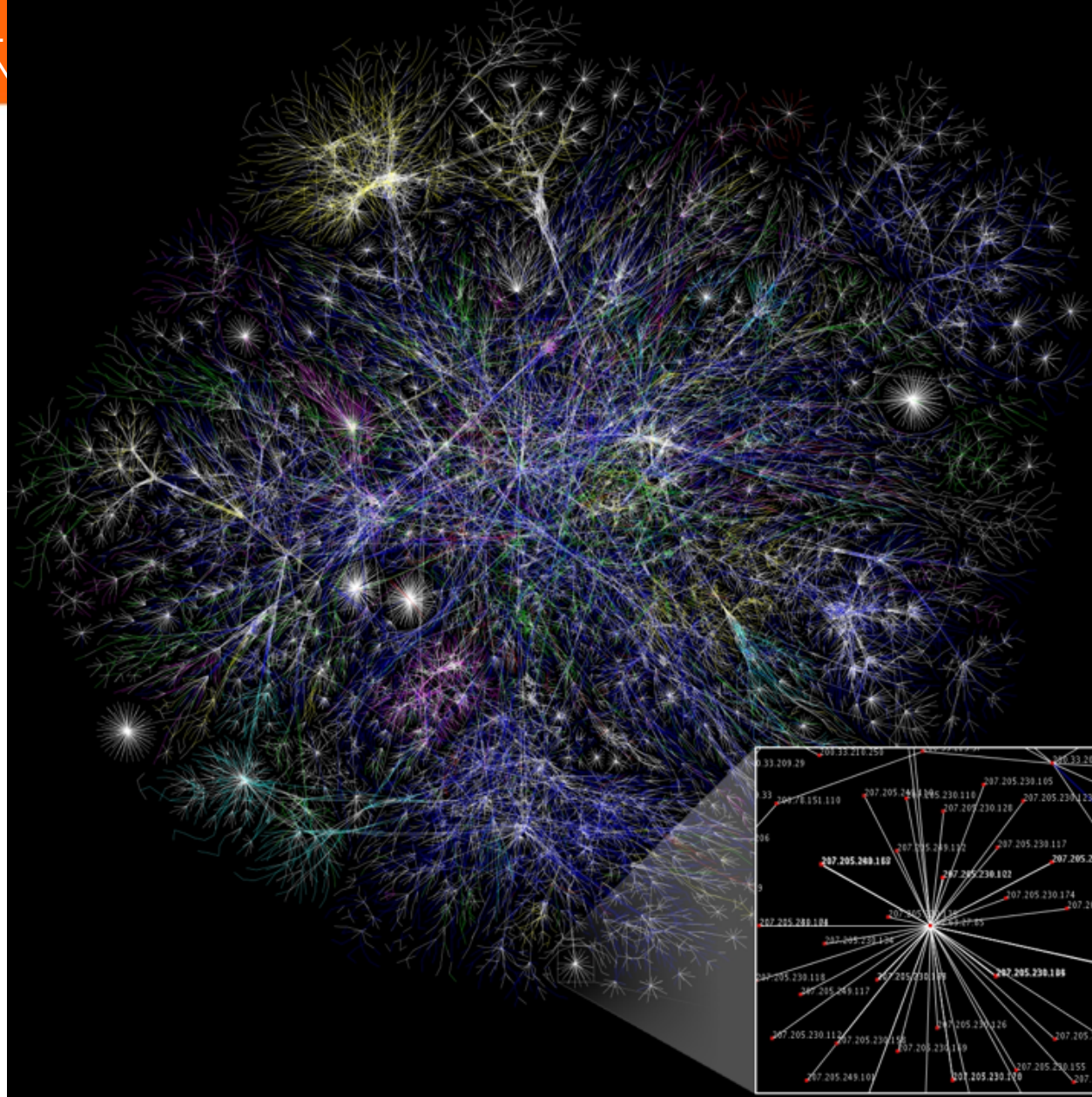
# Ceci est un reseau.

Networks often control critical processes, almost invisibly



# Ceci est un reseau.

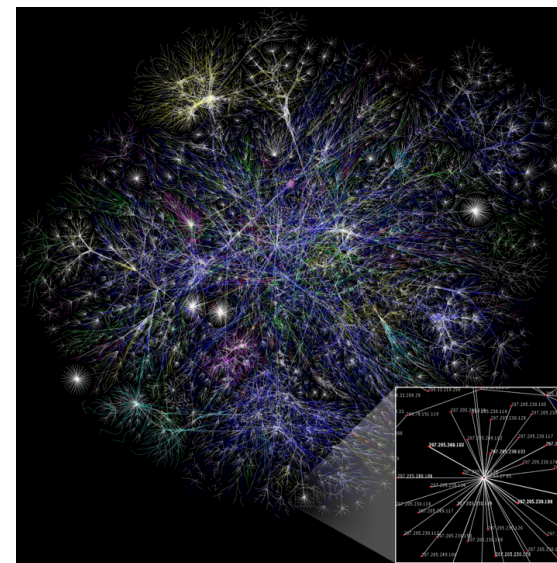
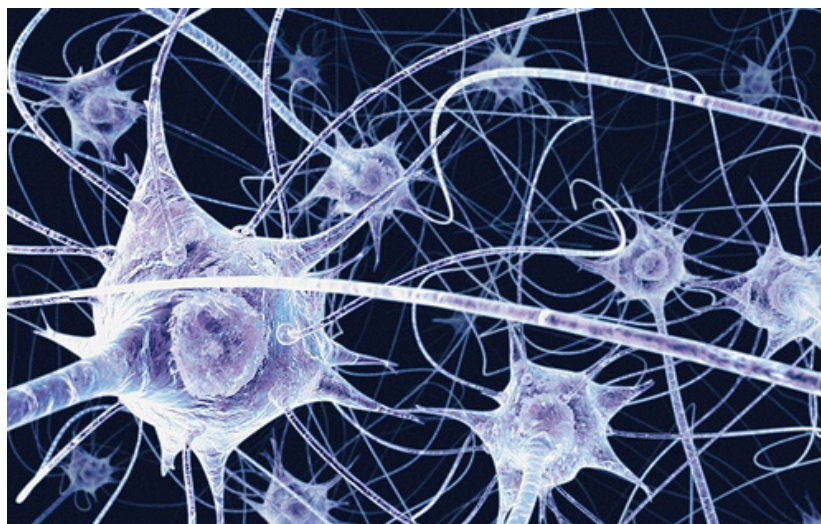
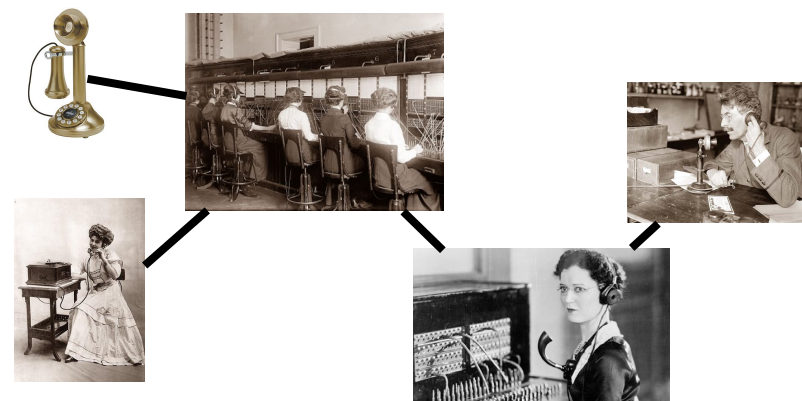
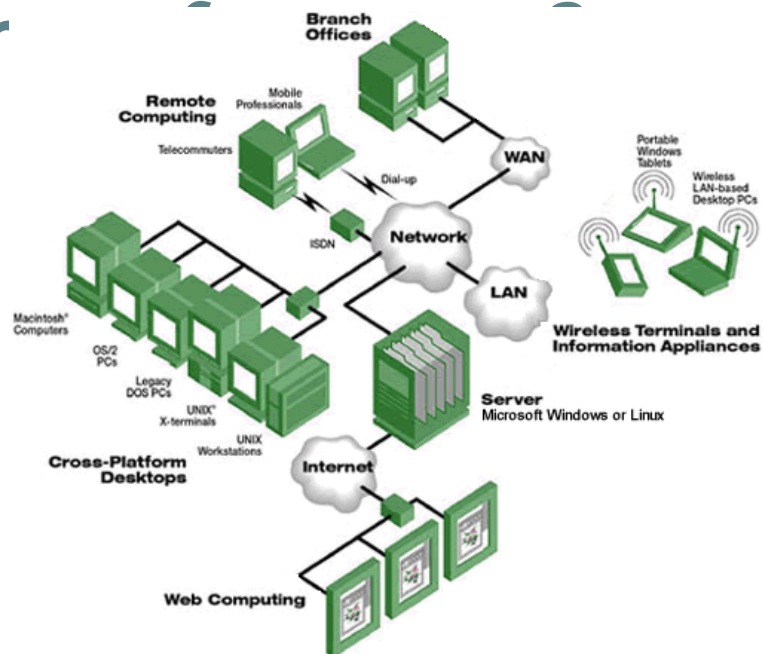
Self-configuring, self-repairing, resilient network.



# Ceci est le Reseau.

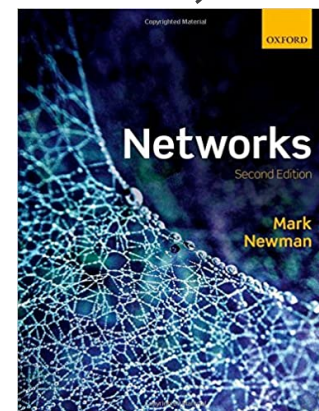
## Picture of the Internet (not even complete)

## Comm



# Common features of networks

- a network is composed by
  - (independent) **nodes**
  - exchanging **informations**
  - using **communication channels**
    - in order to perform a **task** which cannot be done by a single node
- nodes can be computers, people, cells...
- channels can be electrical, optical, chemical, virtual...
- Great book for general networks:  
Mark Newman, “Networks”, 2018, OUP



# Specific features of *computer* networks

- General purpose
  - not optimized for a specific application
  - many applications can coexist
- Openness, flexibility
  - new features can be added dynamically (even not devised at the moment of network design / implementation)
  - vendor-independent

## In this course: general issues and solutions for computer networks

- **requirements** about the services implemented by a network
  - What the applications are expecting from a network?
- **general principles** for implementing, managing and improving these services
  - some theory of signals
  - network architectures
  - protocols
  - some network security
- **main applications and examples**
  - ISO/OSI model, TCP/IPv4 and IPv6 stacks
  - some applications

## What we will not see here

- Specific issues of specific applications
- Most application-level protocols and algorithms
- Suggested courses for continuing on these topics:
  - Distributed Systems
  - Computer Network Security
  - (Tecnologie web per il cloud, IBML second year)

# References

- Suggested book:
  - L. Peterson, B. Davie, “Reti di Calcolatori”, 3<sup>a</sup> ed., Apogeo, 2012.  
Unfortunately it is out of print, but you can find the English (and updated) version at <https://book.systemsapproach.org/>
- Alternatives:
  - A. Tanenbaum, D. Wetherall, “Reti di Calcolatori”, 5<sup>a</sup> ed., Pearson, 2011
- But also: Kurose-Ross, Halsall, Stallings...
- Slides, examples, old exams, general informations: on Teams



# Modalità d'esame

- L'esame si compone di una prova scritta e di una eventuale prova orale.
- La prova scritta richiede di svolgere alcuni esercizi inerenti gli argomenti del Corso e rispondere ad alcune domande teoriche.
- La prova orale è obbligatoria solo per discriminare le “situazioni di confine”:
  - $17 \leq \text{voto scritto} \leq 19$  oppure  $\text{voto scritto} \geq 28$ .
  - Con voto scritto  $< 17$ , non si è ammessi all'orale.
  - ~~Con voto scritto  $< 10$  non ci si può iscrivere all'appello successivo (anche tra sessioni differenti).~~
  - L'orale consiste in due-tre domande e può variare il voto dello scritto al massimo di un paio di punti (in più, ma anche in meno).
- L'esame di Sistemi Operativi è propedeutico. No, non si fanno eccezioni.
- FAQ: <https://marino.miculan.org/faq>



# Chapter 1 Foundations

# Chapter Outline

- In order to understand the design, the implementation and the use of networks, we will see:
  - Example of applications
  - Requirements
  - Network Architecture
  - Implementing Network Software
  - Performance Evaluation

## Chapter Goal

- Exploring the requirements that different applications and different communities place on the computer network
  - users
  - application programmers
  - network operators
  - network and protocol designers
- Introducing the idea of *network architecture*
- Introducing some key elements in implementing Network Software
- Define key metrics that will be used to evaluate the performance of computer network



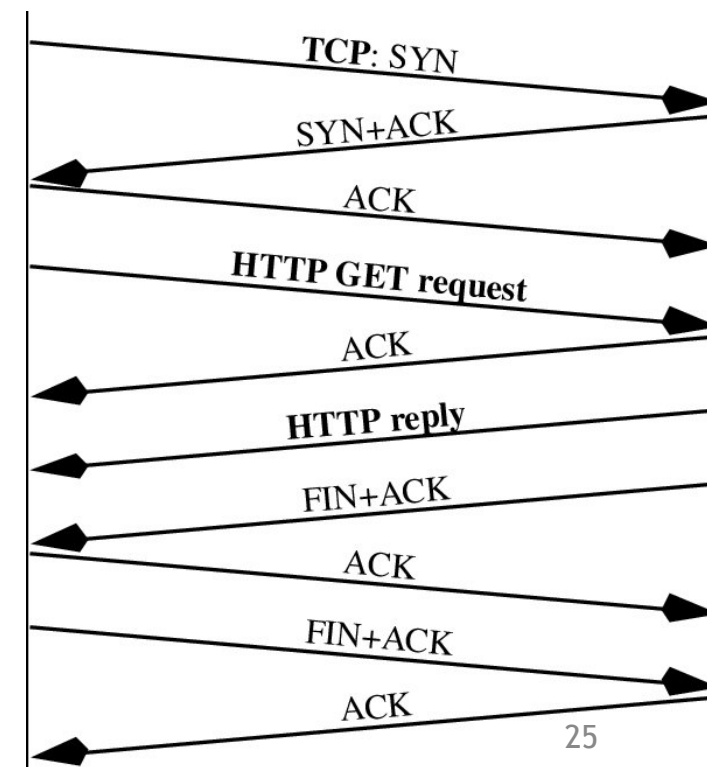
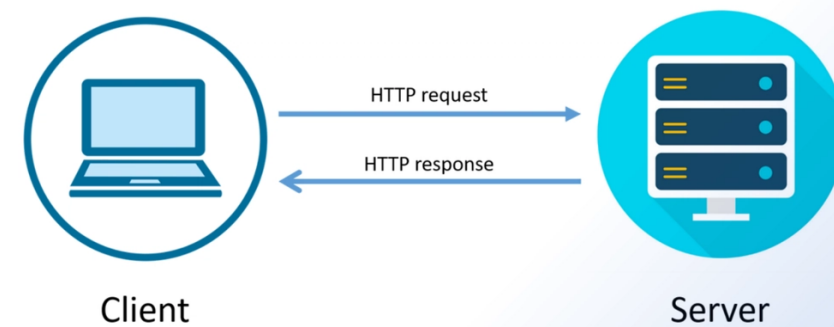
# Applications and Requirements of Computer Networks

# Applications

- Most people know about the Internet (a computer network) through applications
  - World Wide Web
  - Email
  - Online Social Network
  - Streaming Audio Video
  - File Sharing
  - Instant Messaging
  - ...

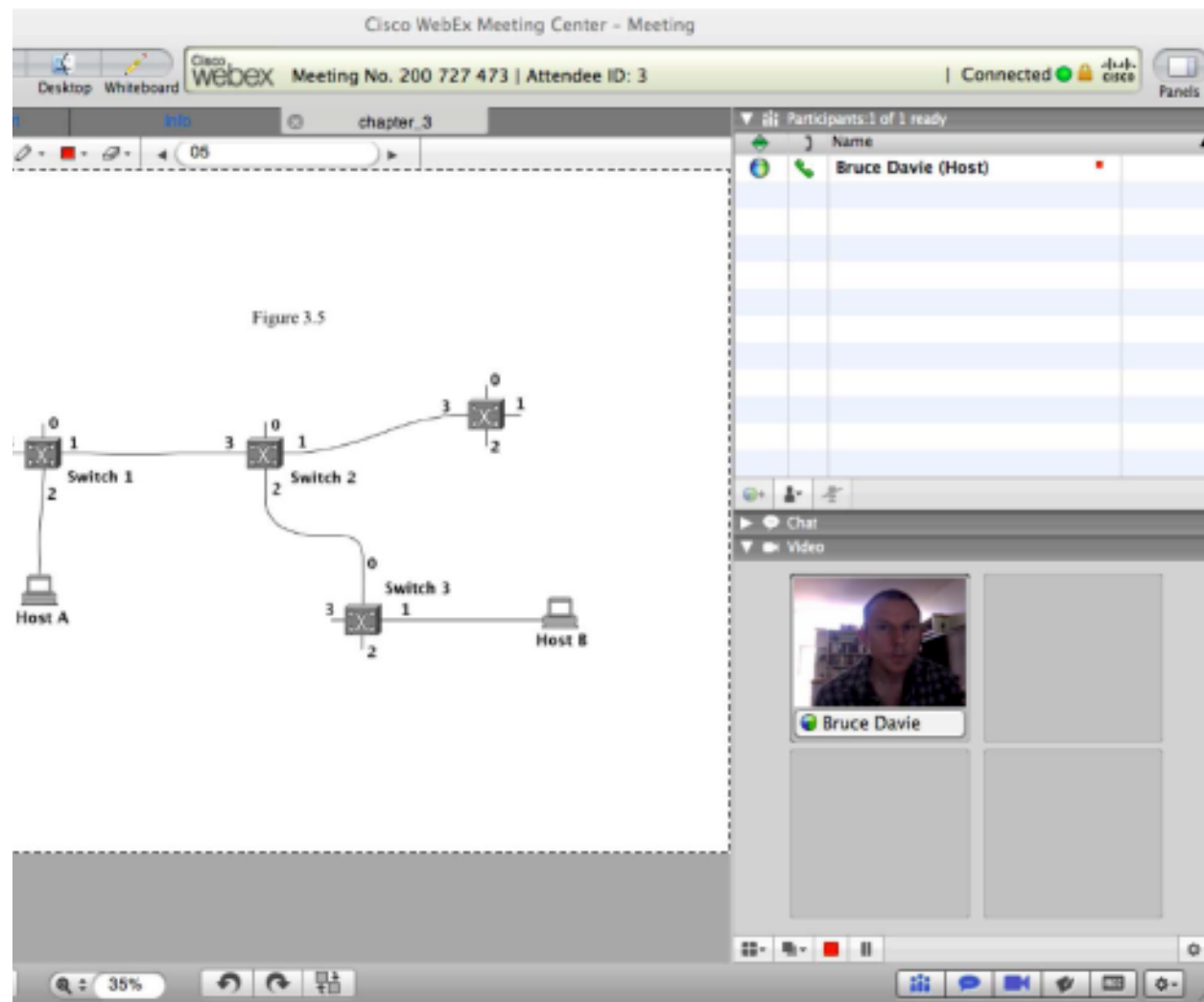
# Example application: WWW

- Main application of Internet for most users
- Hypertext of (possibly dynamically generated) documents, distributed among many servers
- URL: Uniform resource locator
  - <http://www.dimi.uniud.it/miculan>
- HTTP: Hyper Text Transfer Protocol
  - Request: “I need <http://www.dimi.uniud.it/miculan>”
  - Reply: “here is the data you have requested”
- TCP: Transmission Control Protocol:
  - 10 (or more) messages for one URL request
    - 3 for connection establishment of TCP
      - Request: “I got your request and I will send the data”
      - Reply: “Here is the data you requested; I got the data”
    - 4 messages for tearing down TCP connection



# A complete different application: video conferencing

- A multimedia application including video-conferencing
- Composed by several streams
  - Audio (possible multiple)
  - Video (possible multiple)
  - Chat
  - Subtitles
  - Whiteboard ...
- Requires audio-video streaming, hypertext, coordination, etc.

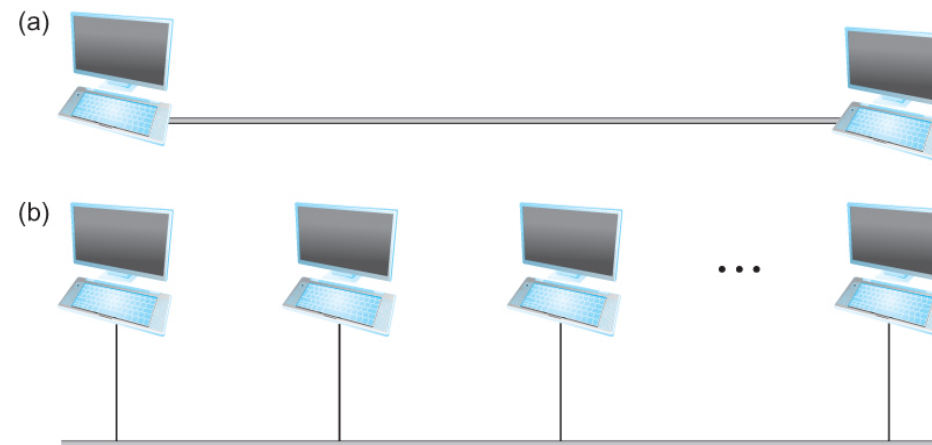


# Requirements

- Important to understand general concepts and basic principles, rather than specific cases
- Different points of view:
  - **Application Programmer**
    - List the services that his application needs: delay bounded delivery of data, mobility, throughput...
  - **Network Designer**
    - Design a cost-effective network with sharable resources, efficient, fair for applications and users...
  - **Network Provider**
    - List the characteristics of a system that is easy to manage

# Connectivity

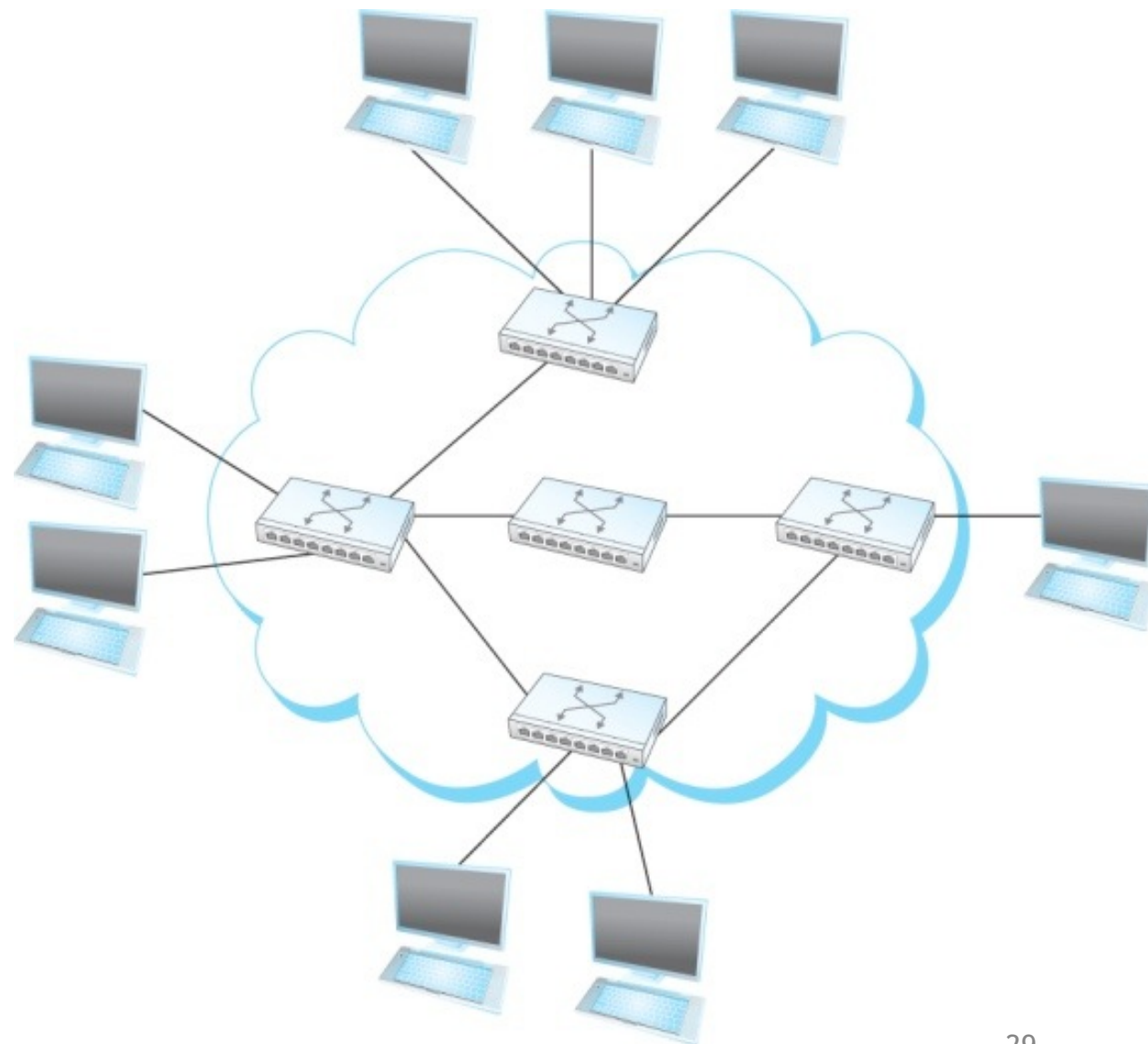
- Need to understand the following terminologies
  - Link
  - Nodes
  - Two main kinds of connections
    - Point-to-point
    - Multiple access
  - Designed to scale?
  - Directed connected networks would be either very limited or very expensive (or both)



(a) Point-to-point  
(b) Multiple access

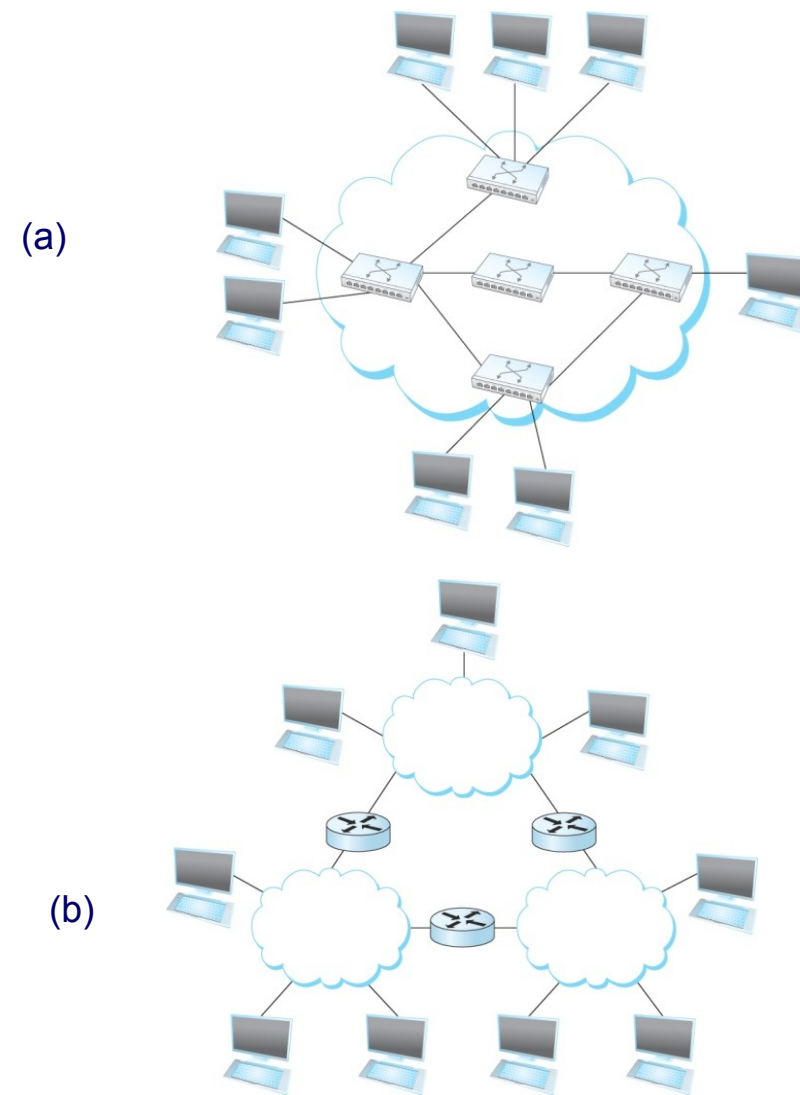
# Connectivity

- Terminology (cont'd)
  - Switched Network
    - Specialized nodes allow for connecting different network trunks
    - Circuit Switched
    - Packet Switched
  - Packet, message
  - Store-and-forward



# Connectivity

- Terminologies (contd.)
  - Cloud
  - Hosts
  - Switches
  - internetwork
- The Internet is an internet
  - Router/gateway
  - Host-to-host connectivity
  - Address
  - Routing
  - Unicast/broadcast/multicast

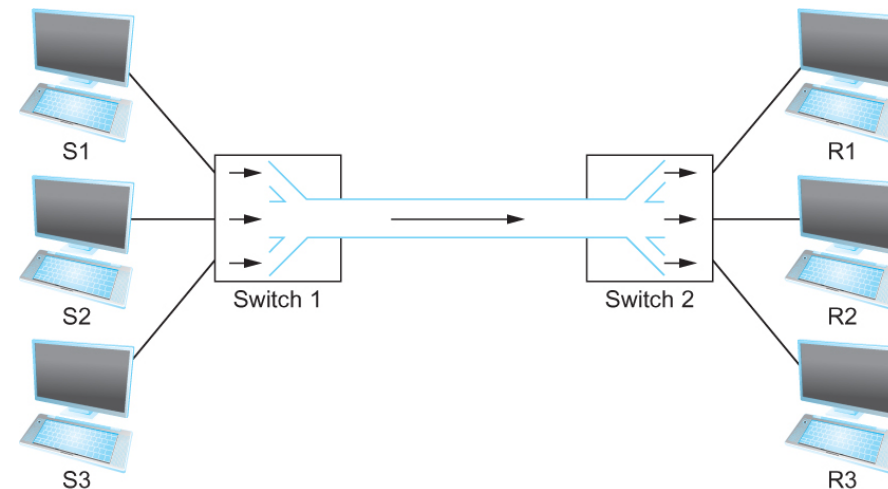


# A recursive definition of network

- A (computer) network is an entity formed by
  - two or more nodes connected by a medium
  - or two or more networks connected by a node.
- **A network is a hierarchy of networks** (often very different)
- Leads to the problem of addressing nodes in different networks, in order to allow message routing and delivery

# Cost-Effective Resource Sharing

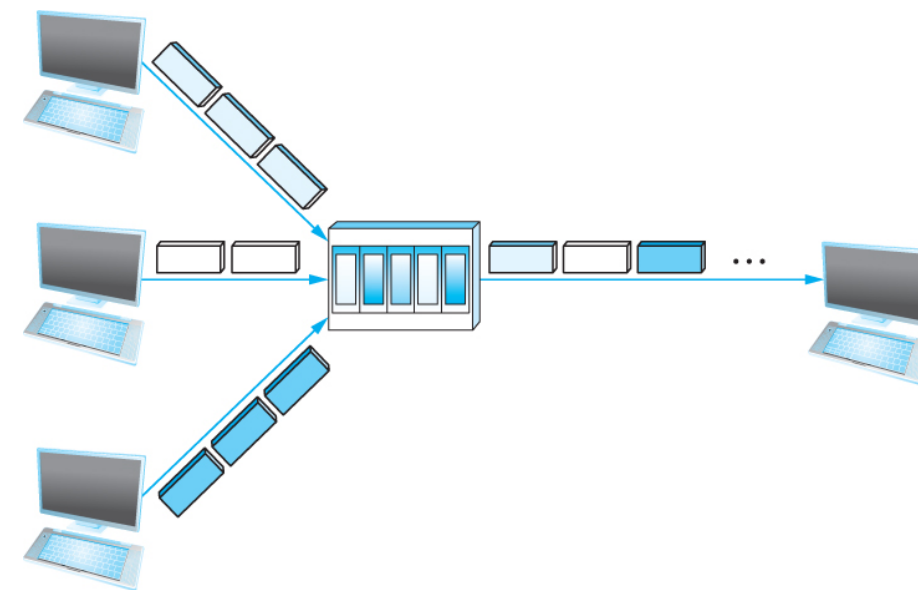
- Important for the network designer and provider
- Resource: links and nodes
- How to share a link?
  - Multiplexing / De-multiplexing
  - Synchronous Time-division Multiplexing
    - Time slots/data transmitted in predetermined slots
  - FDM: Frequency Division Multiplexing
  - Both have limits in efficiency and scalability... so other methods have been developed



Multiplexing multiple logical flows over a single physical link

# Cost-Effective Resource Sharing

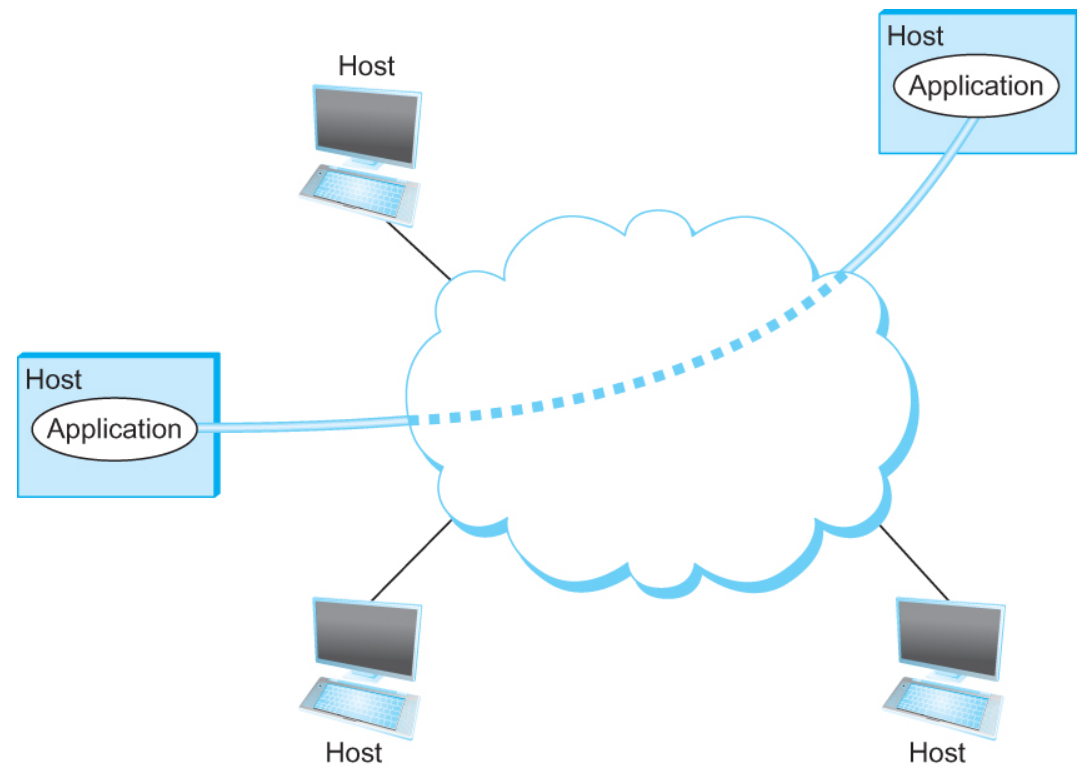
- Statistical Multiplexing
  - Data is transmitted based on demand of each flow.
  - But what is a “flow”?
  - Packets vs. Messages
  - FIFO, Round-Robin, Priorities (Quality-of-Service (QoS))
  - And what if a switch receives more packets than it can handle (i.e. it is congested)?



A switch multiplexing packets from multiple sources onto one shared link

# Support for Common Services

- Programmers do not want to deal with single packets. They need logical channels between applications: application-to-application communication paths or pipes
- The network can be seen as a provider of logical channels, which can be established and maintained on the need
- Next problem is to define the functionalities for these logical channels
  - e.g.: reliable? Message size? Connection or connectionless?



Process communicating over an  
abstract channel

# Common Communication Patterns

- By observing many applications, we can identify common communication patterns
- Client/Server applications are common
  - eg. File transfers/access, name resolutions...
  - But others are possible
- Two basic types of communication channel
  - Request/Reply Channels
    - Suitable for many client/server interactions
    - Can be reliable or not
  - Message Stream Channels
    - Used often for video on demand, video conferences,...
  - But other types are possible

# Reliability

Network should hide the errors

- Bits are lost
  - Bit errors (1 to a 0, and vice versa)
  - Burst errors - several consecutive errors
- Packets are lost (Congestion)
- Links and Node failures
- Messages are delayed
- Messages are delivered out-of-order
- Third parties eavesdrop

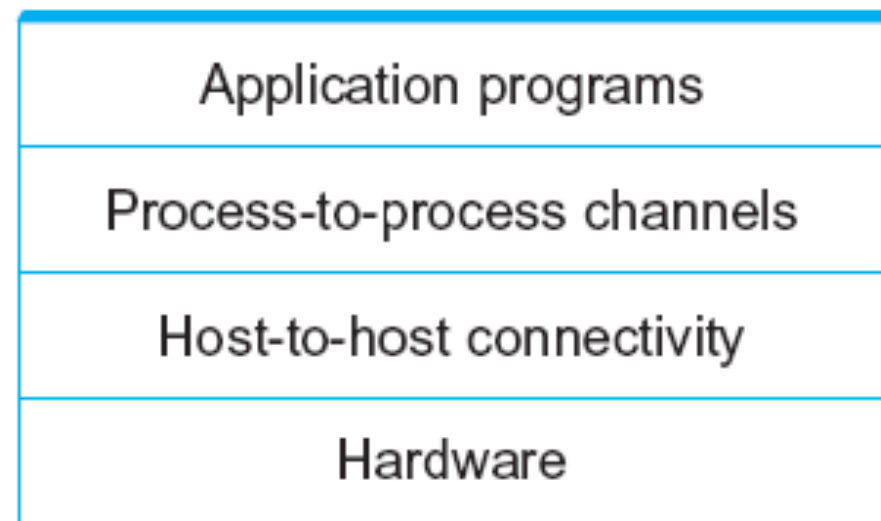
# Management

- Network admins have to manage (configure, monitor, repair) networks easily
- Configuration of large networks cannot be done by a single admin
- Sometimes, networks do not have expert administrators (e.g. home networks, ad hoc networks)
- Networks should be auto configurable, and resilient to failures

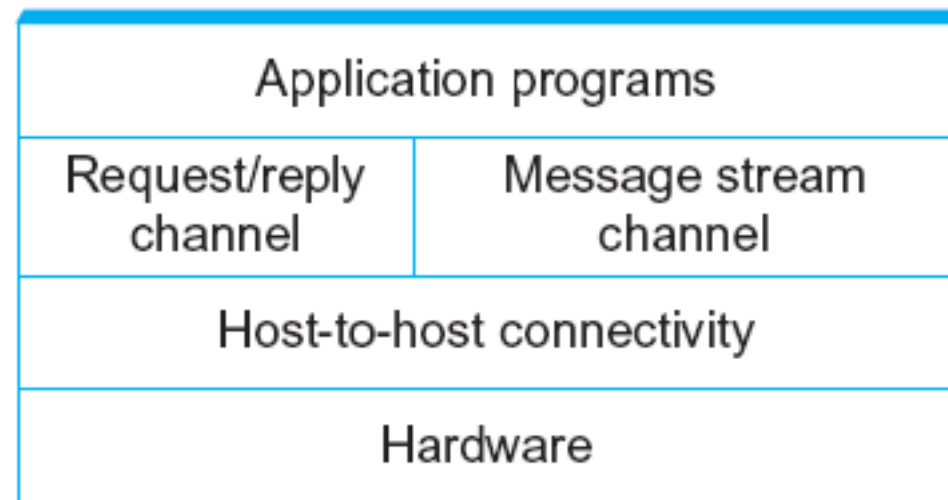
# Network Architecture and Protocols

# Network Architecture

- Networks are complex
- It is difficult to implement all their functions in a single piece of code / hardware
- It is easier to adopt a *layered* architecture (like in layered operating systems)
- Each layer builds on that below it adding new functionalities. Example:

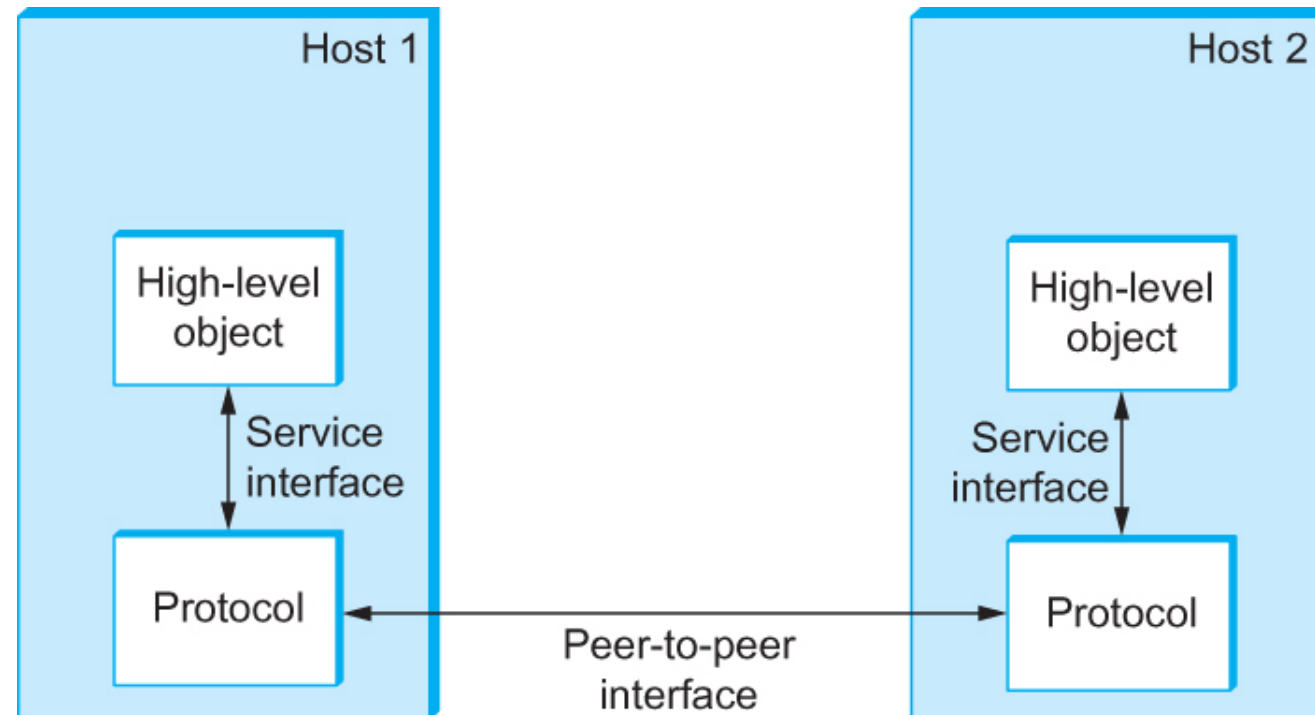


# Network Architecture



Layered system with alternative abstractions available at a given layer

# Interfaces



Service and Peer Interfaces

# Protocols

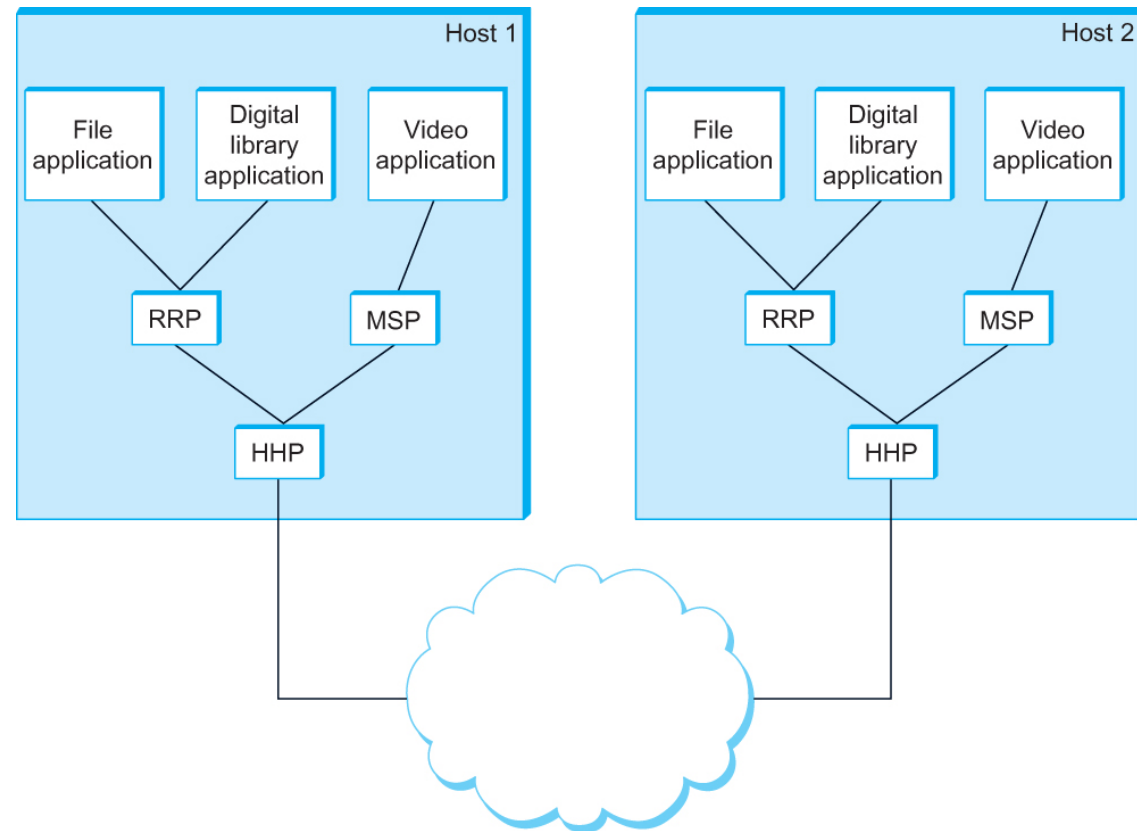
- A **protocol** defines the interfaces between the *layers* in the same system and with the layers of *peer* system
- Building blocks of a network architecture
- Each protocol object has two different interfaces
  - service interface: operations on this protocol
  - peer-to-peer interface: messages exchanged with peer
- Term “protocol” is overloaded
  - specification of peer-to-peer interface
  - module that implements this interface

# Protocols

- Protocol Specification: prose, pseudo-code, state transition diagram
- Interoperable: when two or more protocols that implement the specification accurately
- IETF: Internet Engineering Task Force: defines protocols

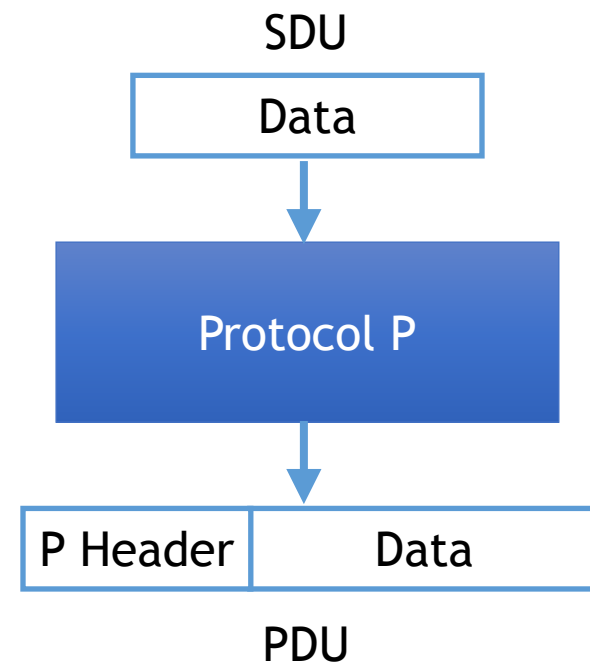
# Protocol Graph

- Example of a protocol graph: nodes are the protocols and links the “depends-on” relation
- RRP: request-reply protocol
- MSP: message stream protocol
- HHP: host-to-host protocol



# Encapsulation

- A fundamental principle of layered architectures
- Each layer needs some information for implementing its functions to upper layers
- These information are carried by some extra data, called the *header*, usually placed in front of the actual message, called the *payload* or *Service Data Unit (SDU)*
- The whole new message is the *Protocol Data Unit (PDU)* of protocol P
- The PDU becomes the SDU of the underlying protocol
- In principle, the protocol does not need to look into the data, but only to its header
- Nor the upper protocols need to access P header



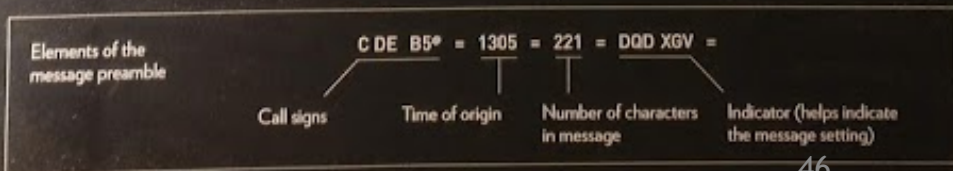
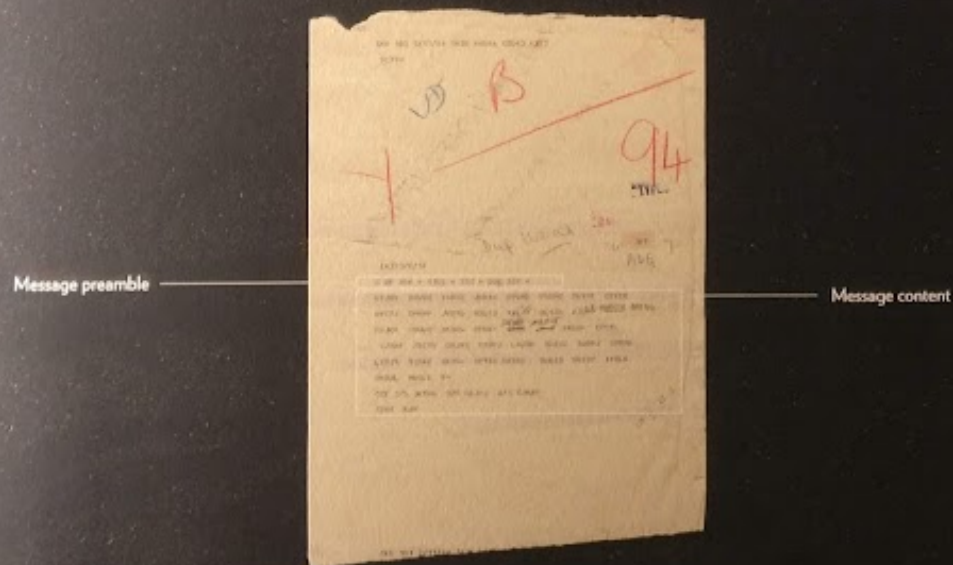
# Encapsulation

- An historical example of encapsulation: Enigma messages from WWII (picture taken at Bletchley Park, UK)

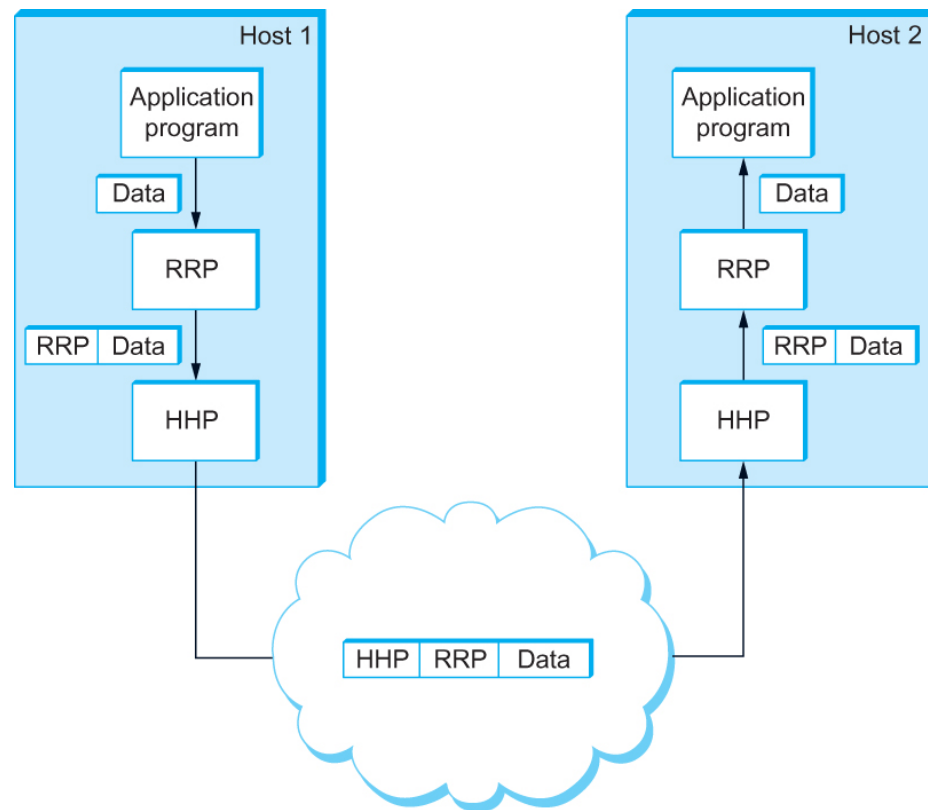
This shows the two main parts of an encrypted Enigma message:

- the message preamble
- the message content.

The message preamble included the sender's call sign, time of transmission, number of characters in the message, and the message setting. It was transmitted before the message content began.

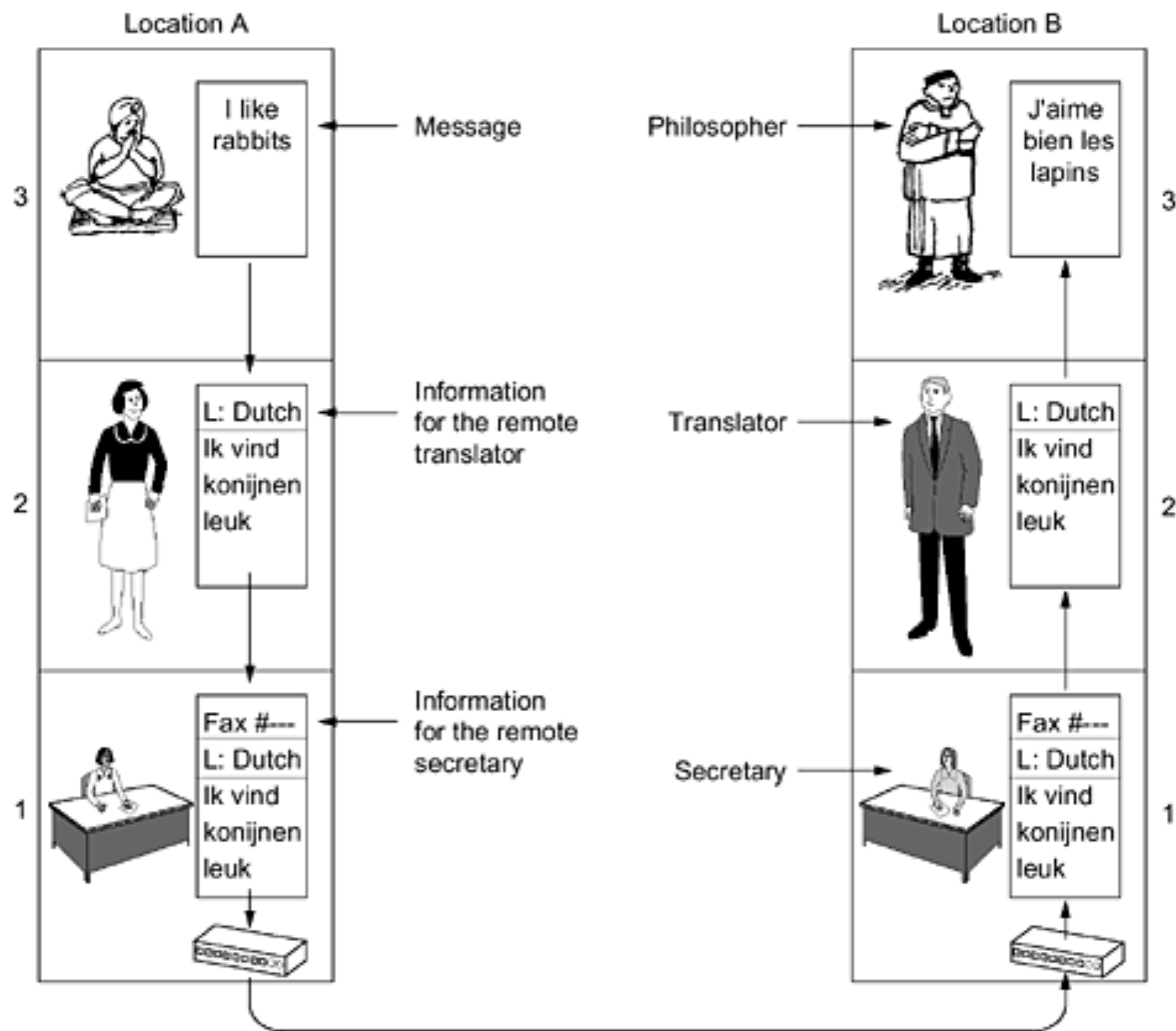


# Encapsulation



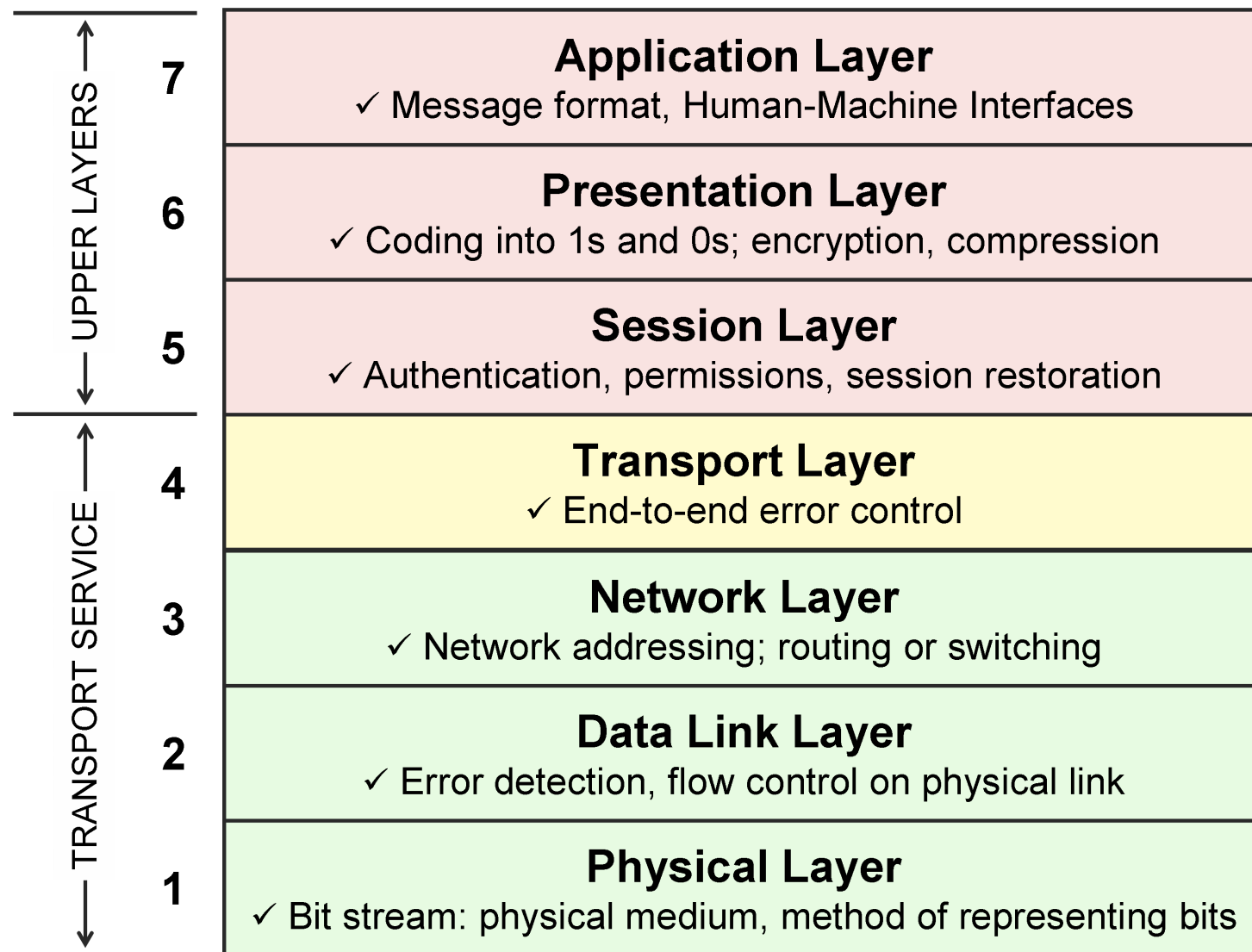
High-level messages are encapsulated inside of low-level messages  
Every layer adds a header containing information for that layer

# Example



# The ISO OSI Architecture

OSI = Open Systems Interconnection



# Description of Lower Layers

## 1. Physical Layer

- Handles the transmission of raw **bits** over a communication link
- Here we deal with encoding/decoding issues

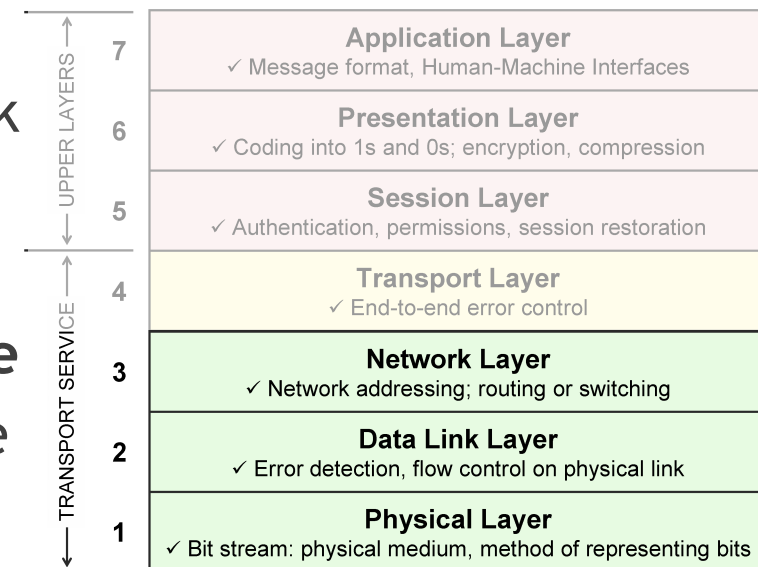
## 2. Data Link Layer

- Collects a stream of bits into a larger aggregate called a **frame**
- Network adaptor along with device driver in OS implement the protocol in this layer
- Frames are actually delivered from host to hosts

## 3. Network Layer

- Handles routing among **nodes** within a packet-switched network
- Unit of data exchanged between nodes in this layer is called a **packet**

- The lower three layers are implemented on all network nodes



# Description of Higher Layers

## 4. Transport Layer

- Implements a process-to-process channel
- Unit of data exchanged in this layer is called **datagram** or **segment**

## 5. Session Layer

- Provides a name space, authentication, coordination and general information among different transport streams that are part of a single application

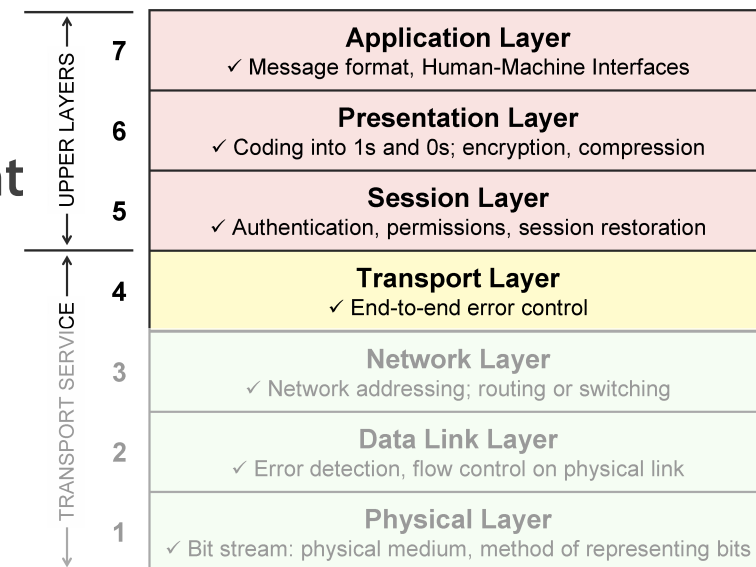
## 6. Presentation Layer

- Concerned about the format of data exchanged between peers
- Data compression, conversion, encryption/decryption ...

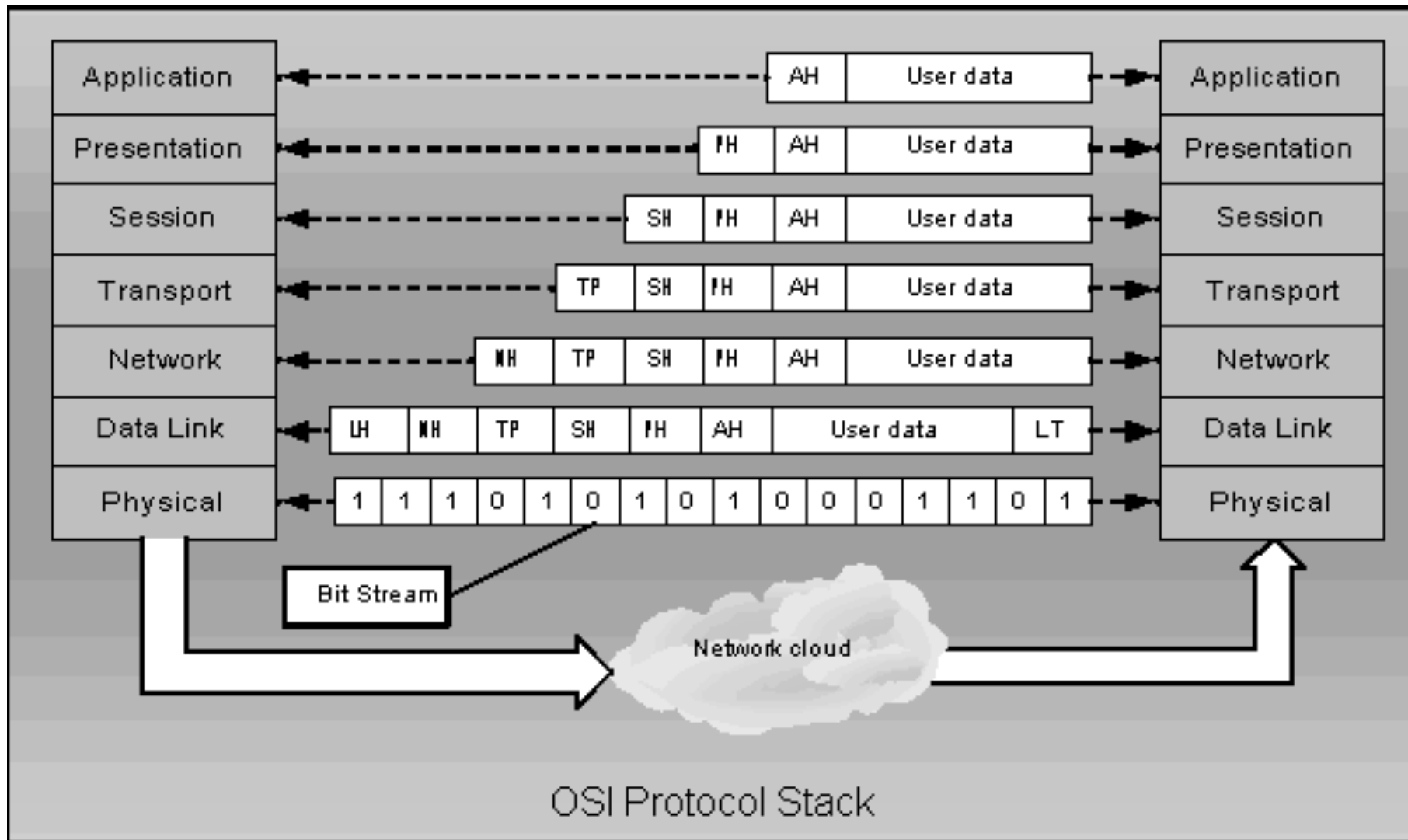
## 7. Application Layer

- Standardize common type of exchanges

- The transport layer and the higher layers typically run only on end-hosts and not on the intermediate switches and routers



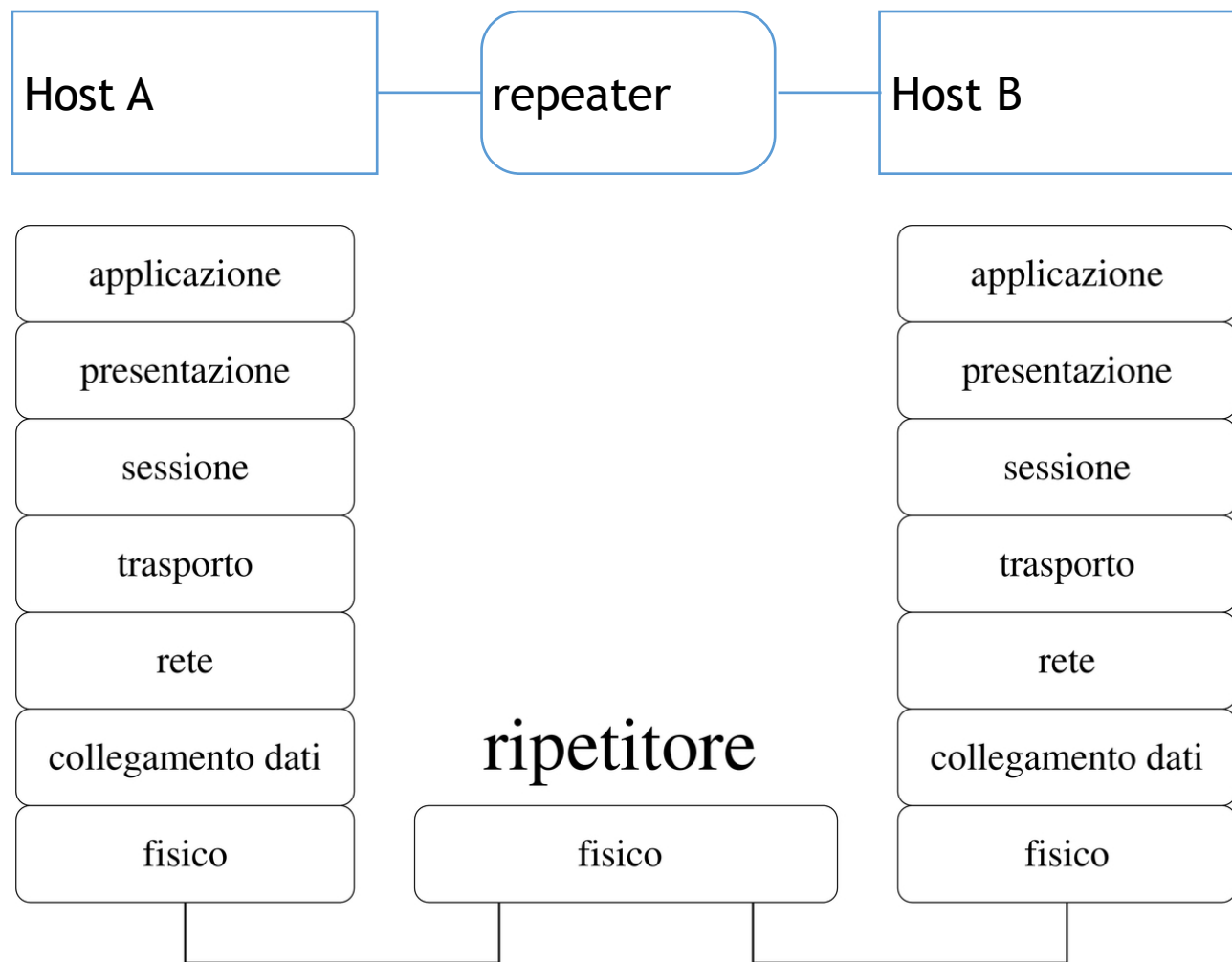
# The ISO OSI Architecture



OSI = Open Systems Interconnection

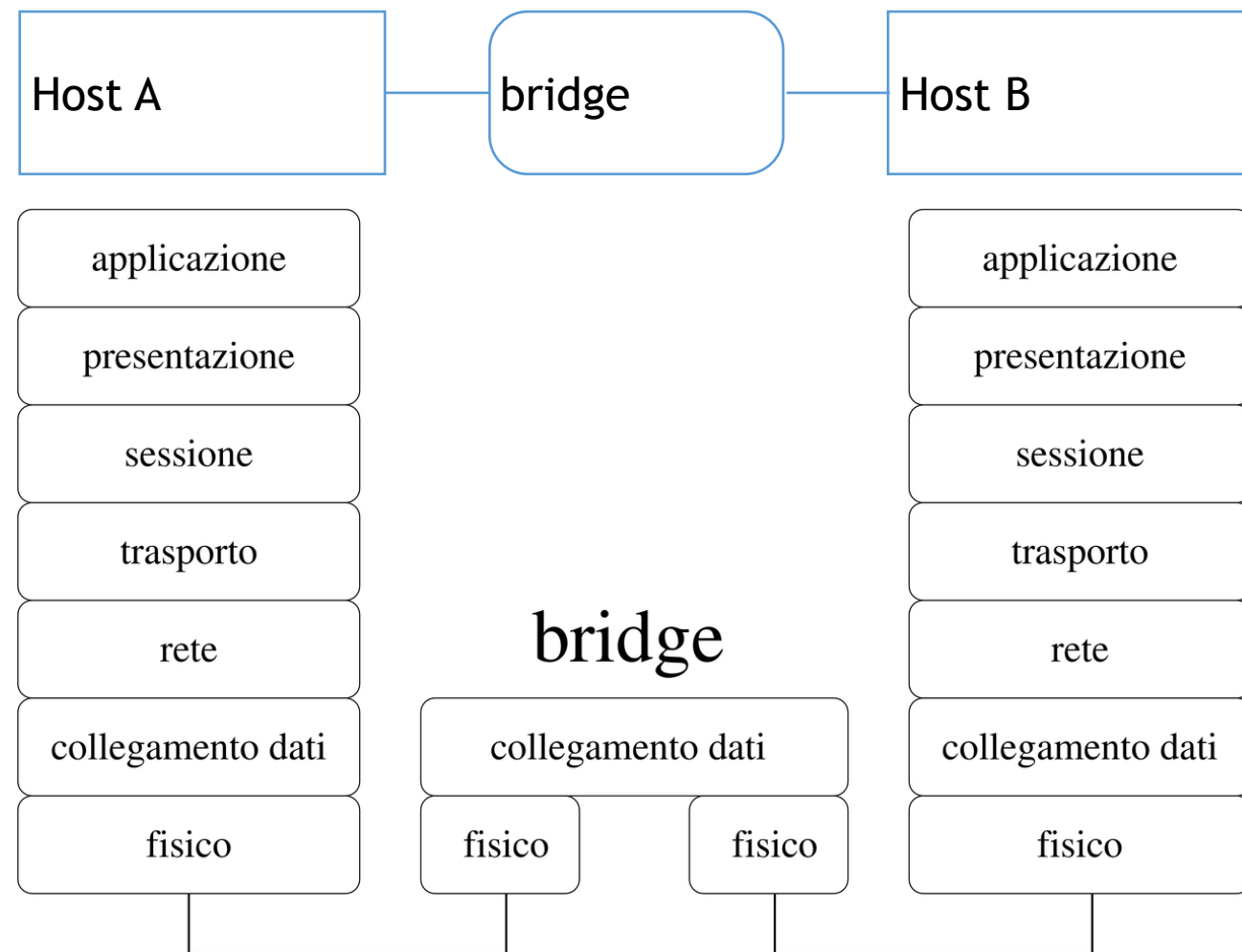
## Repeater = level 1 switch

- Connects two physical media of the same type, which appear as a single one.
- Data units which are switched: single bits.
- Signals are basically the same on the two media, but not simply amplified: bits are decoded from one side and re-encoded on the other side.



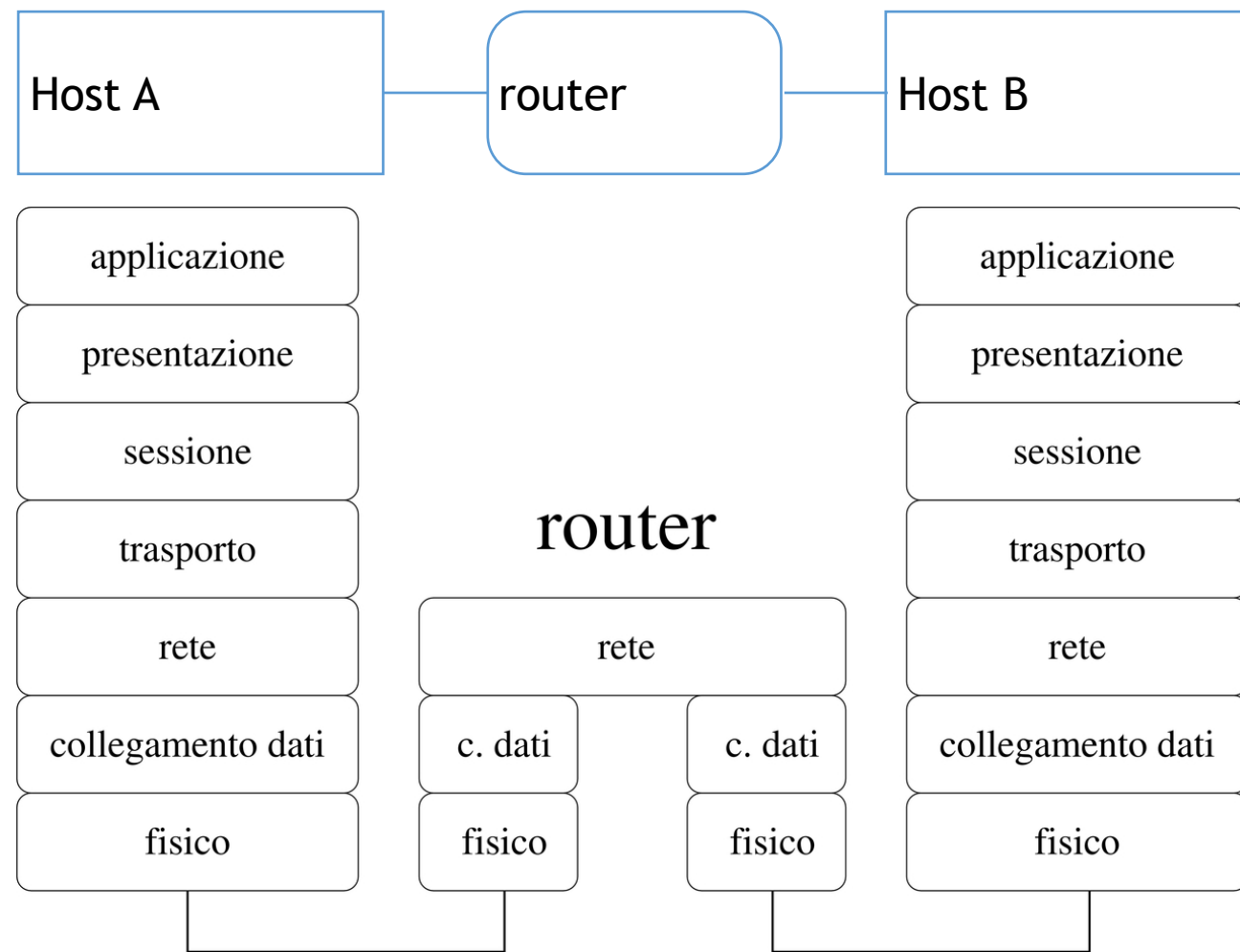
## Bridge = level 2 switch

- Connects two possibly different links, which appear as a single one.
- Data units which are switched: frames. Each frame is received from one side, checked for errors, and forwarded to the other.
- Frame headers are kept unchanged (usually), but physical encodings can differ.
- A *switch* is a multi-port bridge.



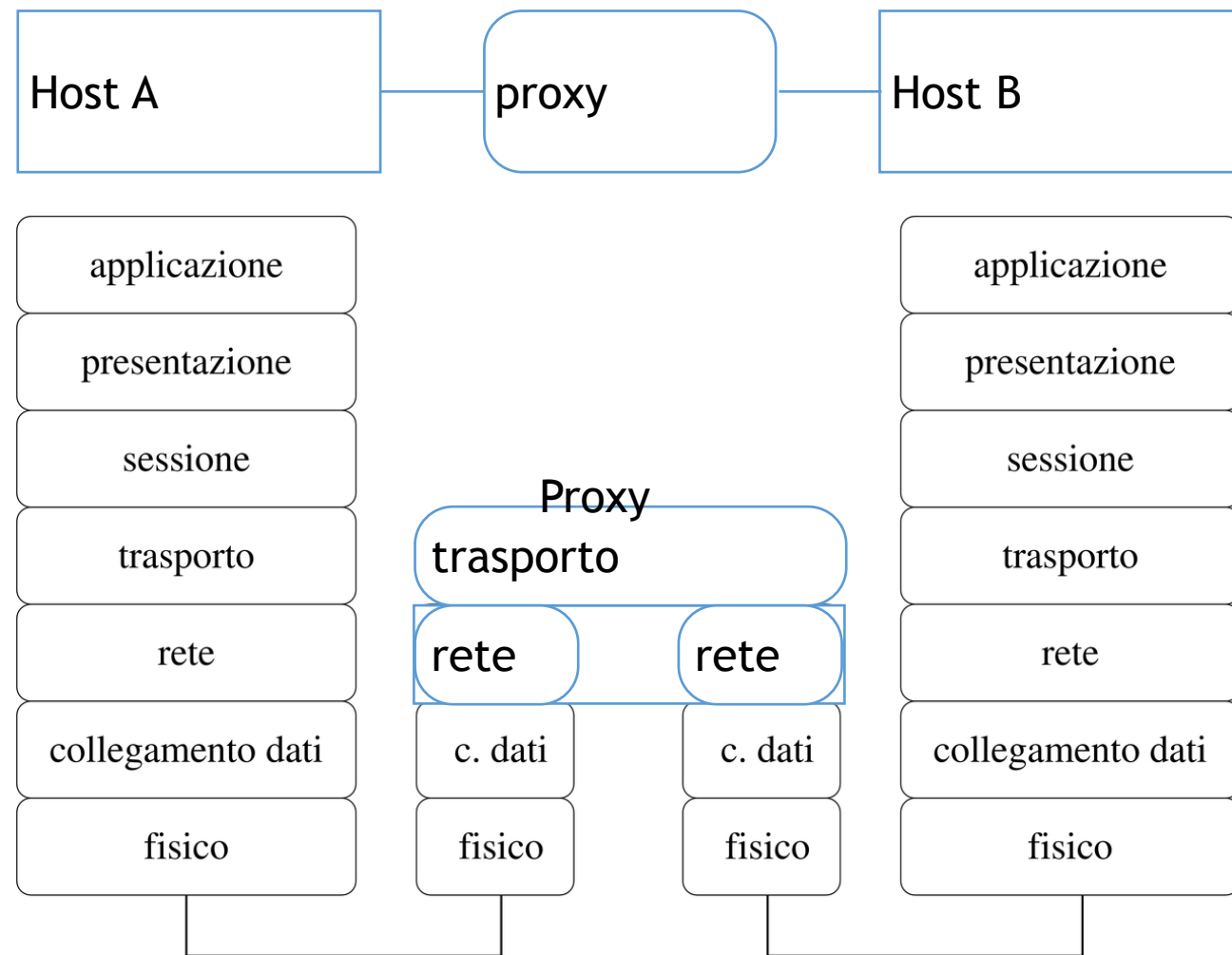
## Router = level 3 switch

- Connects two different network segments, which appear as a single one.
- Data units which are switched: packets.
- Network header is not changed, but frame header is replaced



## Proxy = level 4 switch

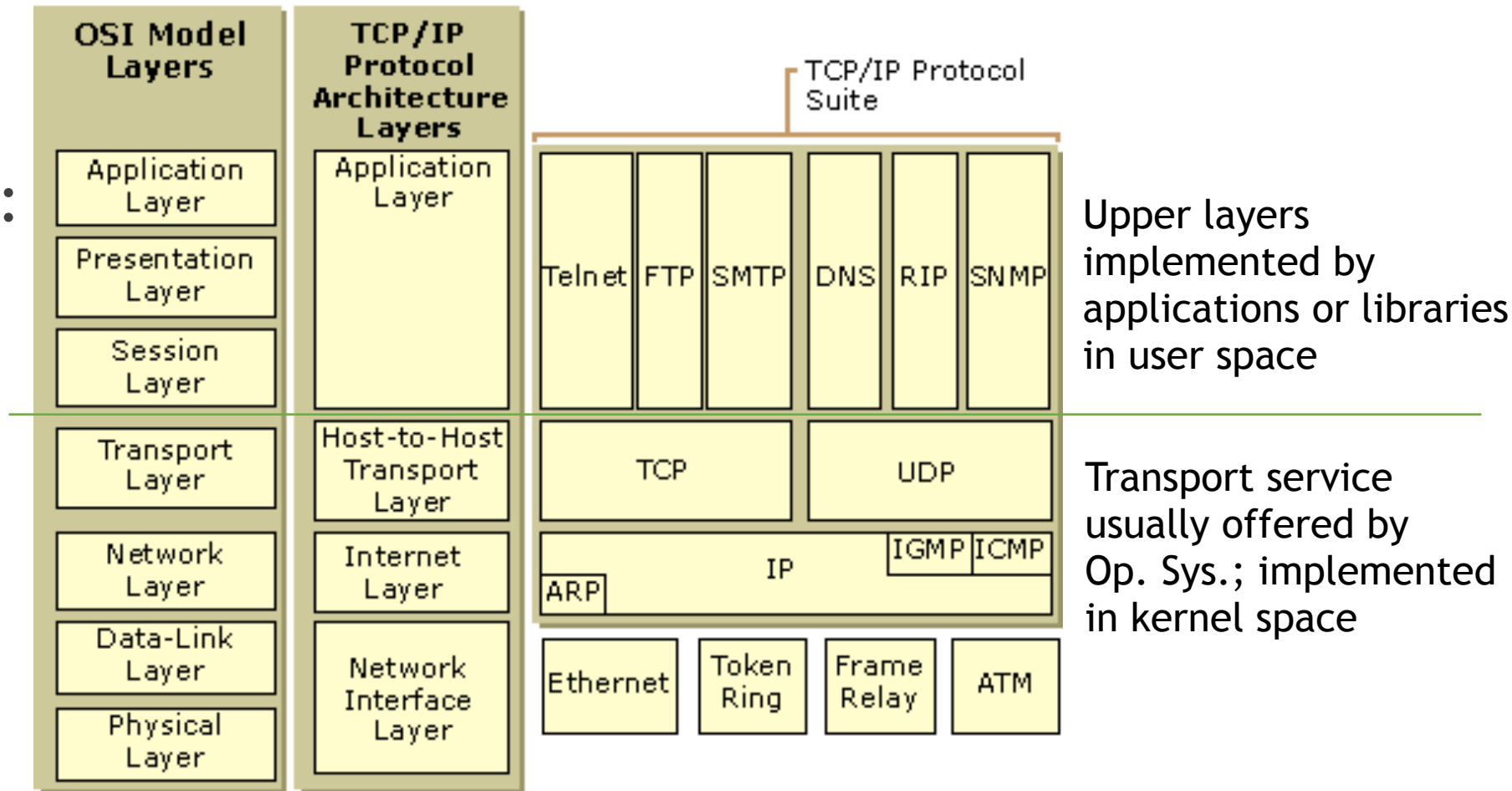
- Translates between different hosts. Data units which are switched: messages.
- Frame and network headers are replaced. Application protocol and transport header are maintained.
- Allows to access a service (B) but connecting to a closer host (the proxy).
- Useful for crossing firewalls, checking traffic content ...



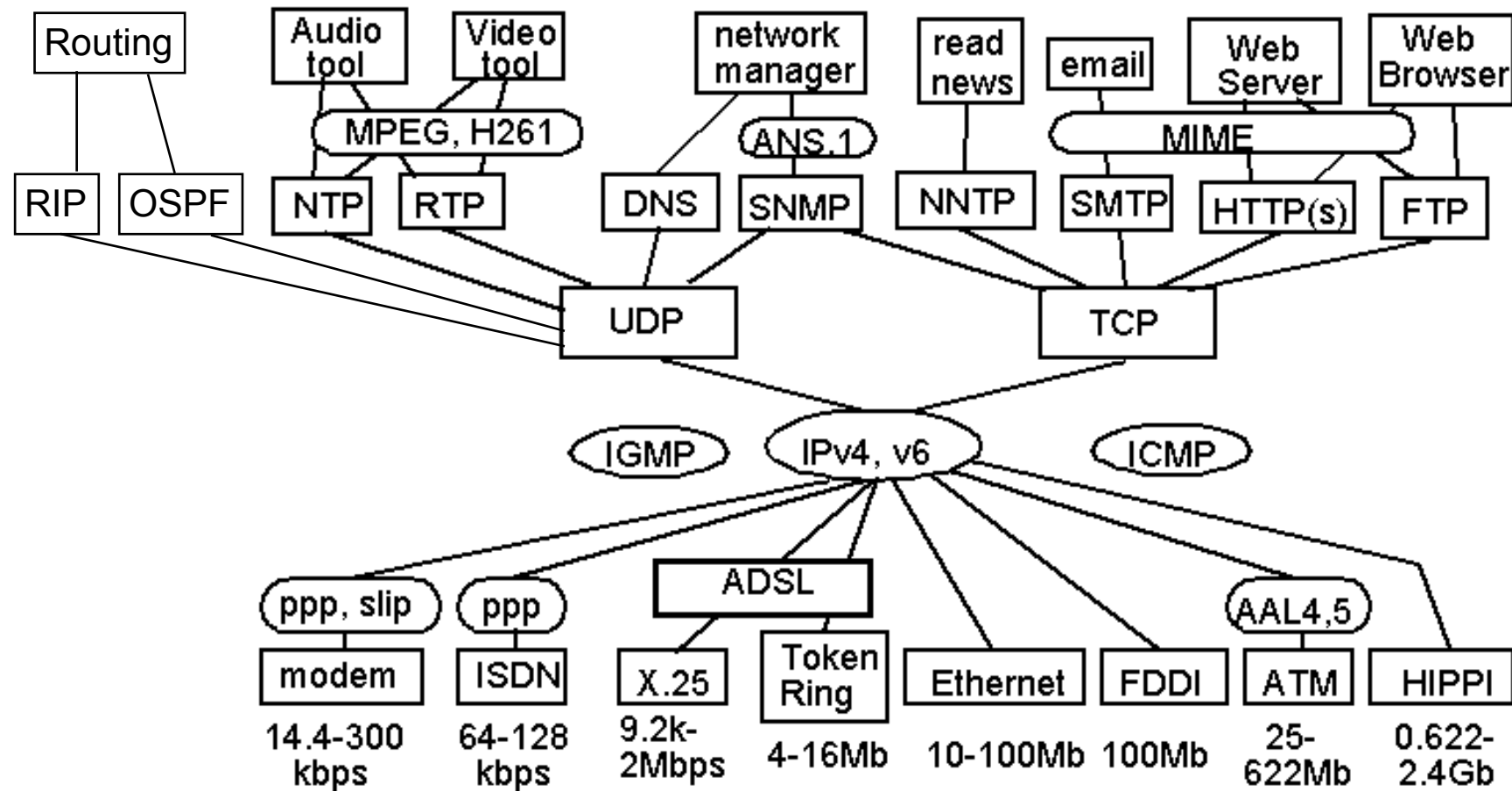
# Internet Architecture (aka the TCP/IP stack)

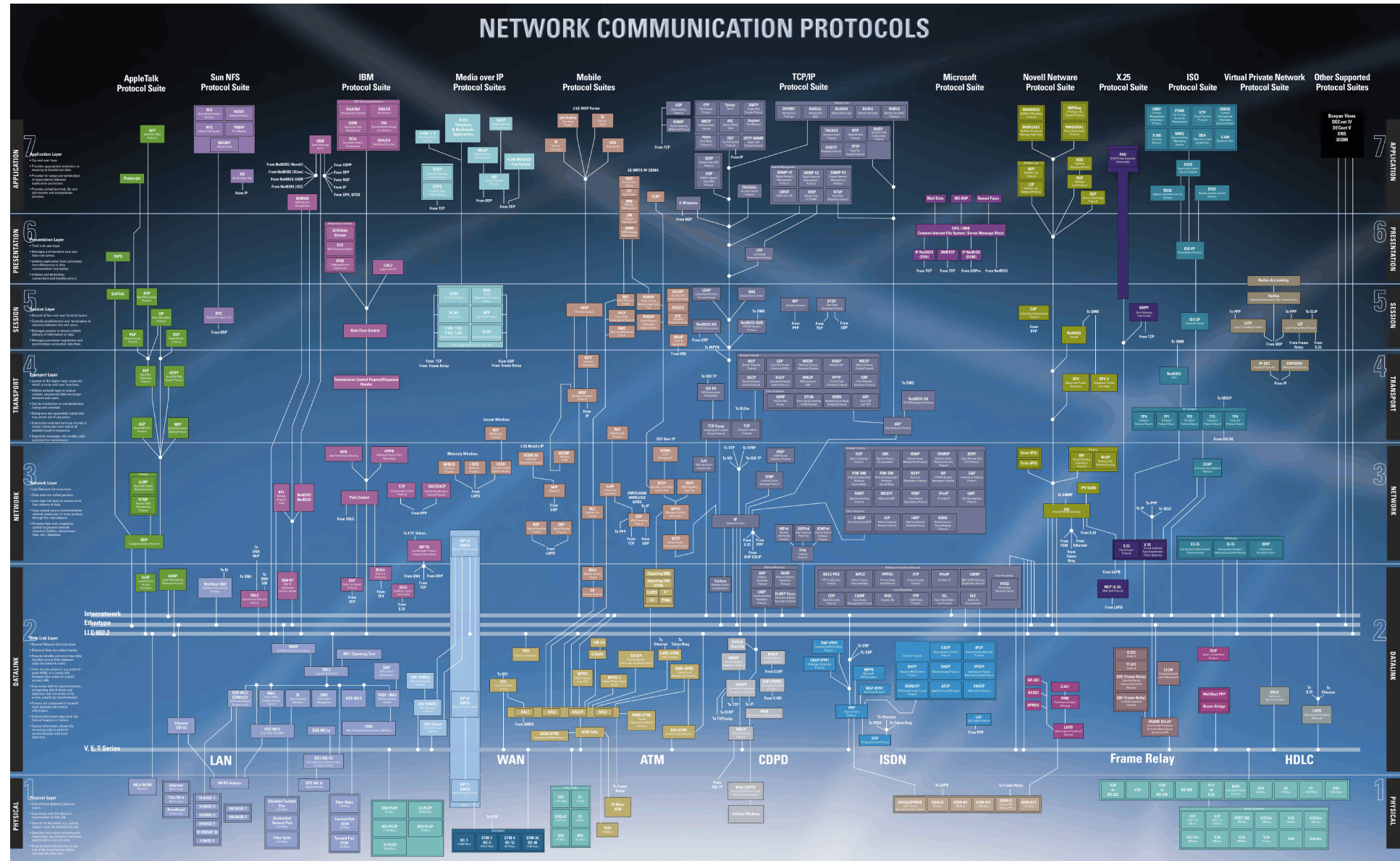
The Internet Architecture is simpler than ISO/OSI:

- Application layer covers Application, Presentation, Session
- Network interface layer covers Data-Link and Physical



# Internet Architecture: protocol graph



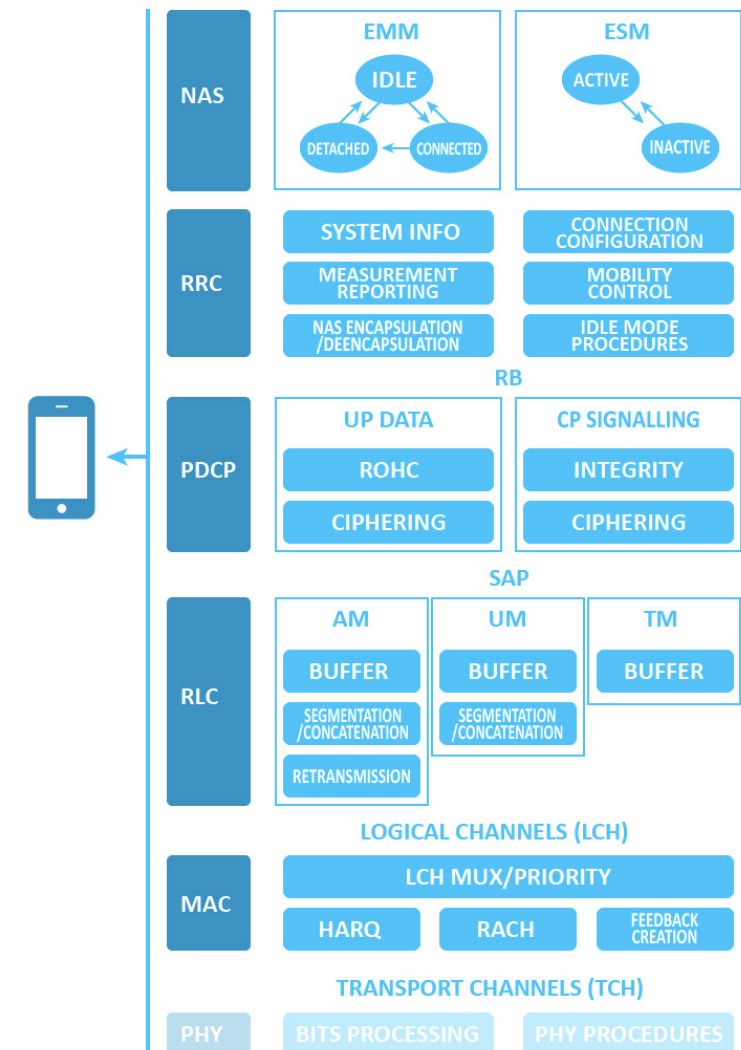


# Internet Architecture

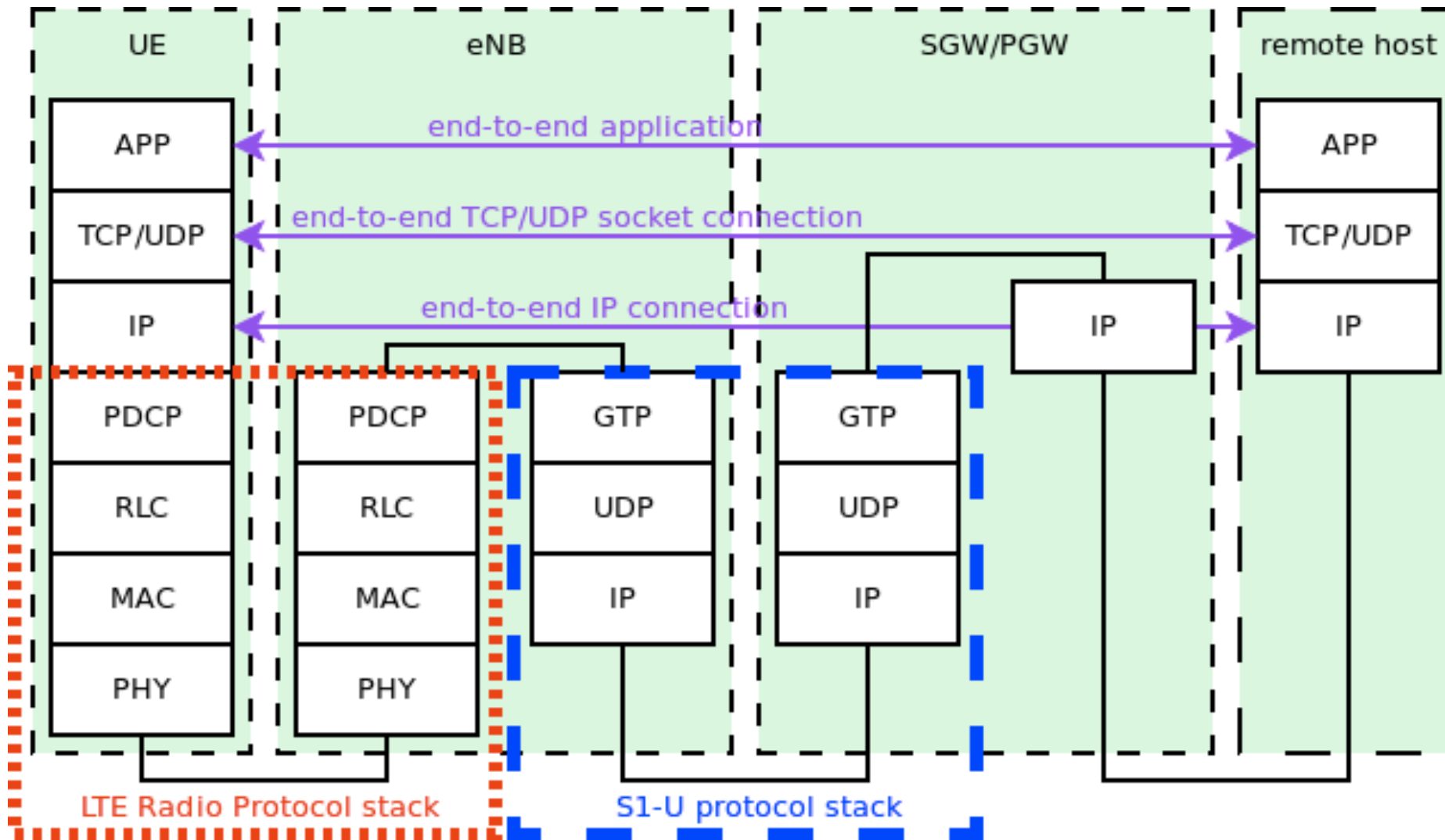
- Defined by IETF
- Three main features
  - **Does not imply strict layering.** The application can bypass the defined transport layers and to directly use IP or other underlying networks
  - An **hour-glass shape** - wide at the top, narrow in the middle and wide at the bottom. IP serves as the focal point for the architecture
  - In order for a new protocol to be officially included in the architecture, it must be provided both a protocol specification and at least one (preferably two) representative implementations

## Other example: LTE-EPC (4G) stack

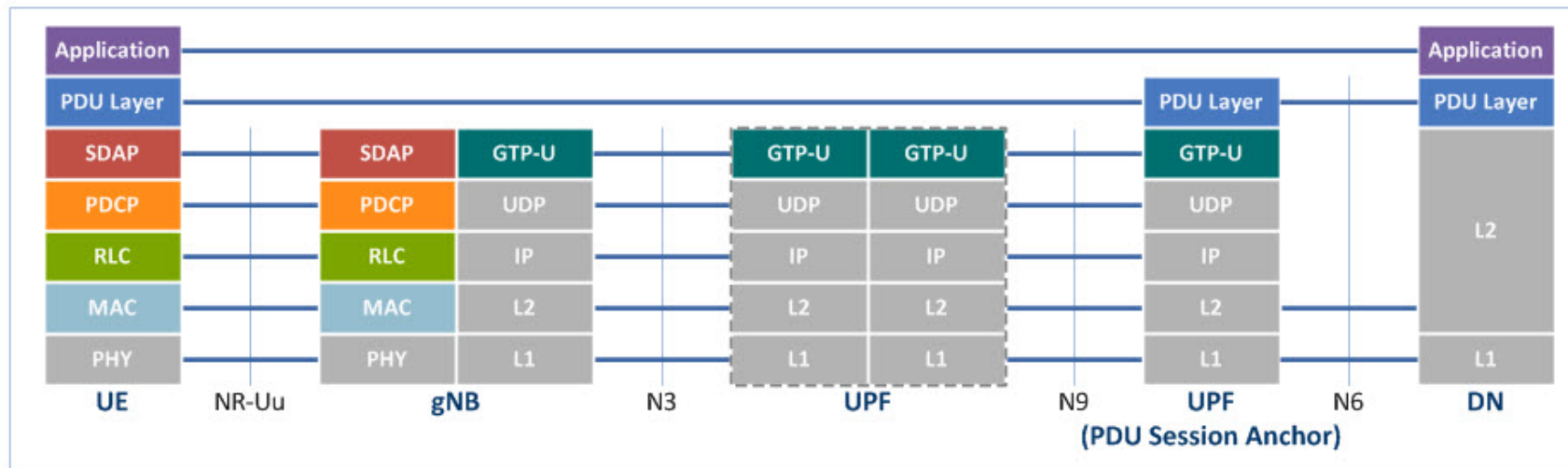
- LTE = Long Term Evolution
- Main goal: “to provide a high data rate, low latency and packet optimized radioaccess technology supporting flexible bandwidth deployments. Its network architecture has been designed with the goal to support packet-switched traffic with seamless mobility and great quality of service.”



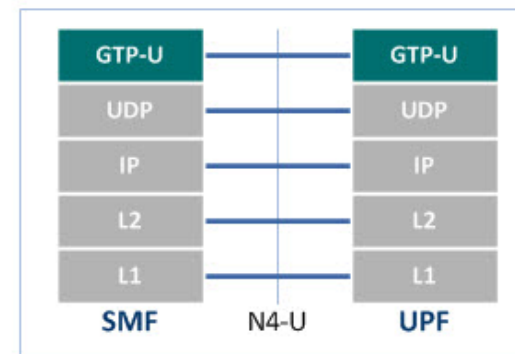
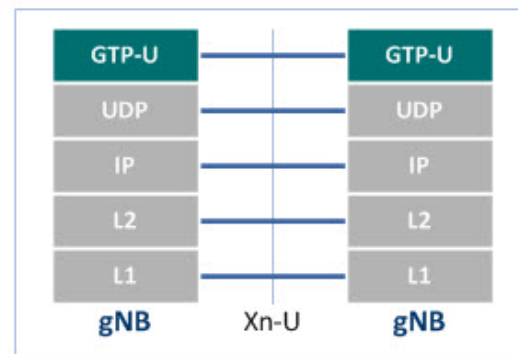
## Other example: LTE-EPC (4G) stack



# Other example: 5G stack



PDU Layer: IP, Ethernet, etc.



DN : Data Network  
gNB : Next generation NodeB  
GTP-U : GPRS Tunneling Protocol User plane  
MAC : Medium Access Control  
PDCP : Packet Data Convergence Protocol  
PDU : Protocol Data Unit

RLC : Radio Link Control  
SDAP : Service Data Adaptation Protocol  
SMF : Session Management Function  
UE : User Equipment  
UPF : User Plane Function  
Xn-U : Xn User plane

## Stratified architectures: limits

- Not all services can fit easily in some layer
- Some services are *inherently* cross-layer
  - **Security services**
  - **Quality of service**
- In fact, these are “modalities”, i.e. these describe how other services are offered
- In these cases, we implement them in one, some or even **all** layers...



# Network API

# Application Programming Interface

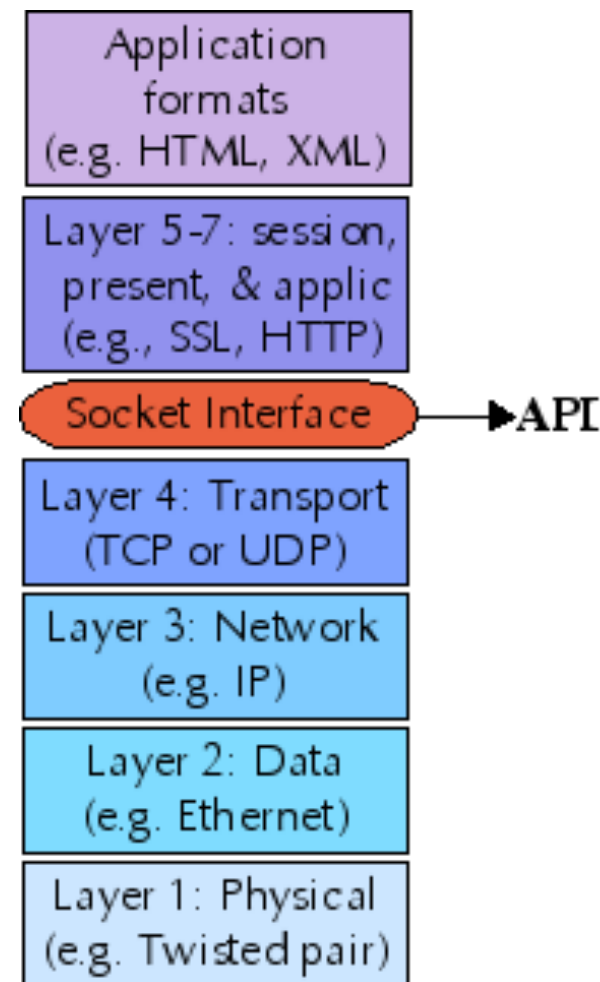
- Interface exported by the network
- Since most network protocols are implemented in software and nearly all computer systems implement their network protocols as part of the operating system, when we refer to the interface “exported by the network”, we are generally referring to the interface that the OS provides to its networking subsystem
- The interface is called the **network Application Programming Interface (API)**

## Application Programming Interface (Sockets)

- **Socket Interface** was originally provided by the Berkeley distribution of Unix
  - Now supported in virtually all operating systems, and found in any programming language
  - Some languages implement higher-level interfaces on top of sockets
- Each protocol provides a certain set of services, and the API provides a syntax by which those services can be invoked in this particular OS

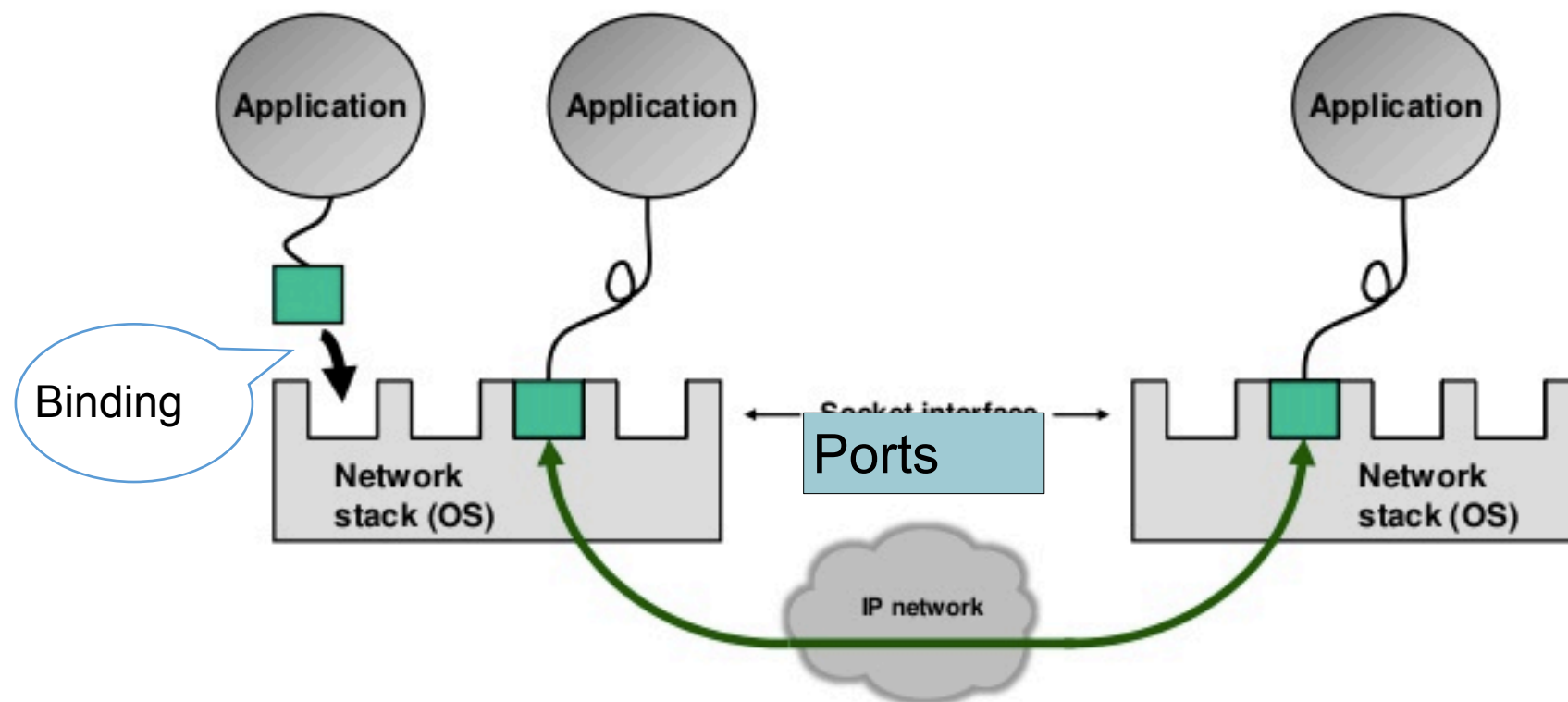
# Socket

- What is a socket?
  - The point where a local application process attaches to the network
  - An interface between an application and the network
- The interface defines operations for
  - Creating a socket
  - Attaching a socket to the network
  - Sending and receiving messages through the socket
  - Closing the socket



# Stream oriented Sockets

- After connection, the channel is bidirectional.



# Socket creation

At creation the application specifies:

- Socket Family
  - PF\_INET denotes the Internet family
  - PF\_UNIX denotes the Unix pipe facility
  - PF\_PACKET denotes direct access to the network interface (i.e., it bypasses the TCP/IP protocol stack)
- Socket Type
  - SOCK\_STREAM is used to denote a byte stream: reliable, ordered, connection-oriented
  - SOCK\_DGRAM denotes a message oriented service, usually unreliable, such as that provided by UDP

## Creating a Socket

```
int sockfd = socket(address_family, type, protocol);
```

- The socket number returned is the socket descriptor for the newly created socket

```
int sockfd = socket (PF_INET, SOCK_STREAM, 0);
```

```
int sockfd = socket (PF_INET, SOCK_DGRAM, 0);
```

- The combination of PF\_INET and SOCK\_STREAM implies TCP

# Client-Server Model with TCP

- Server
  - Passive open
  - Prepares to accept connection, does not actually establish a connection
- Server invokes

```
int bind (int socket, struct sockaddr *address, int
addr_len)

int listen (int socket, int backlog)

int accept (int socket, struct sockaddr *address,
int *addr_len)
```

# Client-Server Model with TCP

- Bind
  - Binds the newly created socket to the specified address i.e. the network address of the local participant (the server)
  - Address is a data structure which combines IP and port
- Listen
  - Defines how many connections can be pending on the specified socket
- Accept
  - Carries out the passive open
  - Blocking operation
    - Does not return until a remote participant has established a connection
    - When it does, it returns a new socket that corresponds to the new established connection and the address argument contains the remote participant's address

# Client-Server Model with TCP

- Client
  - Application performs active open
  - It says who it wants to communicate with
- Client invokes

```
int connect (int socket, struct sockaddr *address,  
            int addr_len)
```
- Connect
  - Does not return until TCP has successfully established a connection at which application is free to begin sending data
  - Address contains remote machine's address

# Client-Server Model with TCP

- In practice
  - The client usually specifies only remote participant's address and let's the system fill in the local information
  - Whereas a server usually listens for messages on a well-known port
  - A client does not care which port it uses for itself, the OS simply selects an unused one

## Client-Server Model with TCP

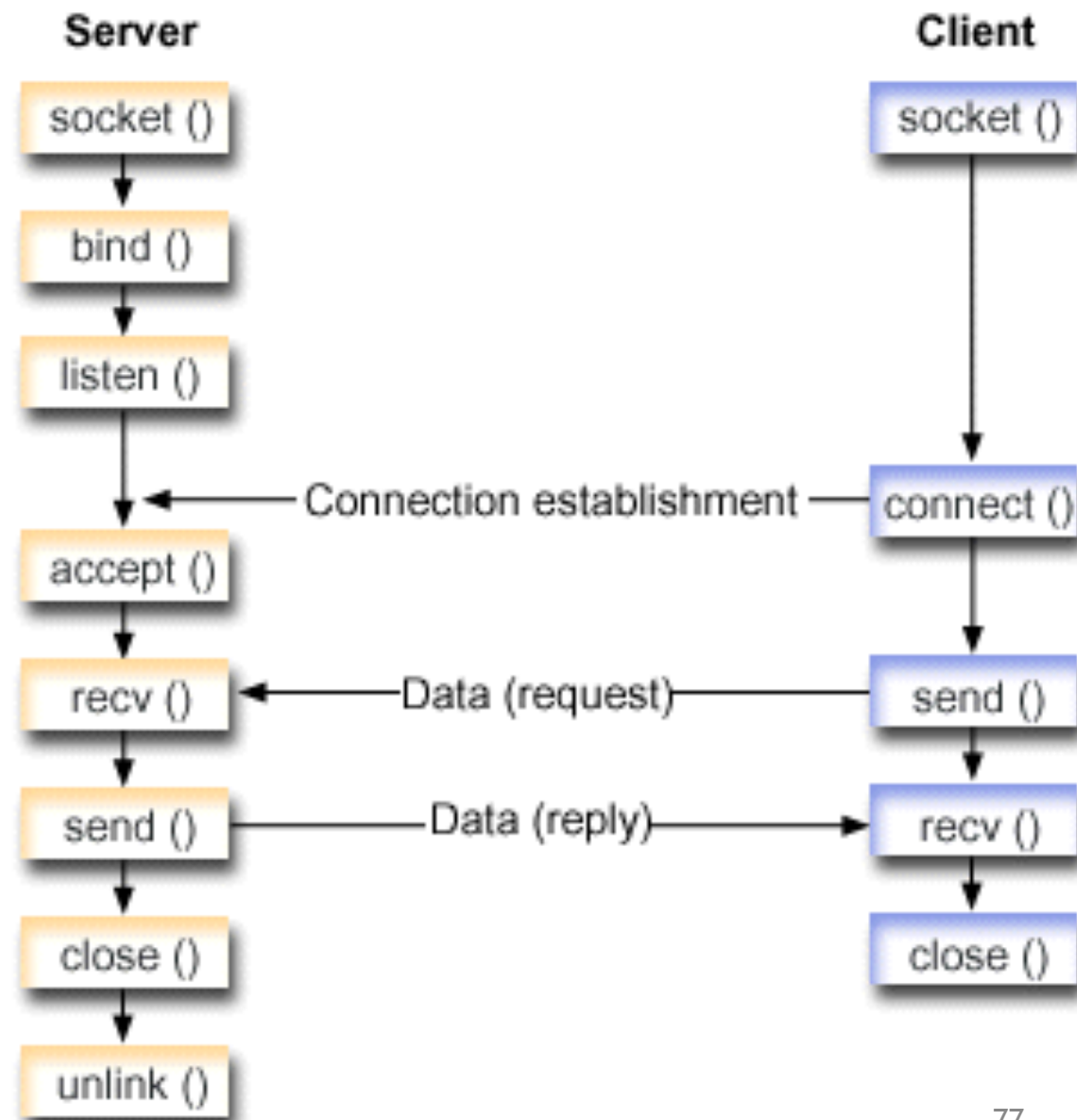
- Once a connection is established, the application process invokes two operation

```
int send (int socket, char *msg, int msg_len, int flags)
```

```
int recv (int socket, char *buff, int buff_len, int flags)
```

- (but also other operations are available)

# Client-Server Model with TCP



# Example Application: Client

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 5432
#define MAX_LINE 256

int main(int argc, char * argv[])
{
    FILE *fp;
    struct hostent *hp;
    struct sockaddr_in sin;
    char *host;
    char buf[MAX_LINE];
    int s;
    int len;
    if (argc==2) {
        host = argv[1];
    }
    else {
        fprintf(stderr, "usage: simplex-talk host\n");
        exit(1);
    }
}
```

```
/* translate host name into peer's IP address */
hp = gethostbyname(host);
if (!hp) {
    fprintf(stderr, "simplex-talk: unknown host: %s\n", host);
    exit(1);
}
/* build address data structure */
bzero((char *)&sin, sizeof(sin));
sin.sin_family = AF_INET;
bcopy(hp->h_addr, (char *)&sin.sin_addr, hp->h_length);
sin.sin_port = htons(SERVER_PORT);
/* active open */
if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0) {
    perror("simplex-talk: socket");
    exit(1);
}
if (connect(s, (struct sockaddr *)&sin, sizeof(sin)) < 0) {
    perror("simplex-talk: connect");
    close(s);
    exit(1);
}
/* main loop: get and send lines of text */
while (fgets(buf, sizeof(buf), stdin)) {
    buf[MAX_LINE-1] = '\0';
    len = strlen(buf) + 1;
    send(s, buf, len, 0);
}
}
```

# Example Application: Server

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#define SERVER_PORT 5432
#define MAX_PENDING 5
#define MAX_LINE 256

int main()
{
    struct sockaddr_in sin;
    char buf[MAX_LINE];
    int len;
    int s, new_s;
    /* build address data structure */
    bzero((char *)&sin, sizeof(sin));
    sin.sin_family = AF_INET;
    sin.sin_addr.s_addr = INADDR_ANY;
    sin.sin_port = htons(SERVER_PORT);

    /* setup passive open */
    if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0) {
        perror("simplex-talk: socket");
        exit(1);
    }
```

```
    if ((bind(s, (struct sockaddr *)&sin, sizeof(sin))) < 0) {
        perror("simplex-talk: bind");
        exit(1);
    }
    listen(s, MAX_PENDING);
    /* wait for connection, then receive and print text */
    while(1) {
        if ((new_s = accept(s, (struct sockaddr *)&sin, &len)) < 0) {
            perror("simplex-talk: accept");
            exit(1);
        }
        while (len = recv(new_s, buf, sizeof(buf), 0))
            fputs(buf, stdout);
        close(new_s);
    }
}
```



# Performance

# Performance

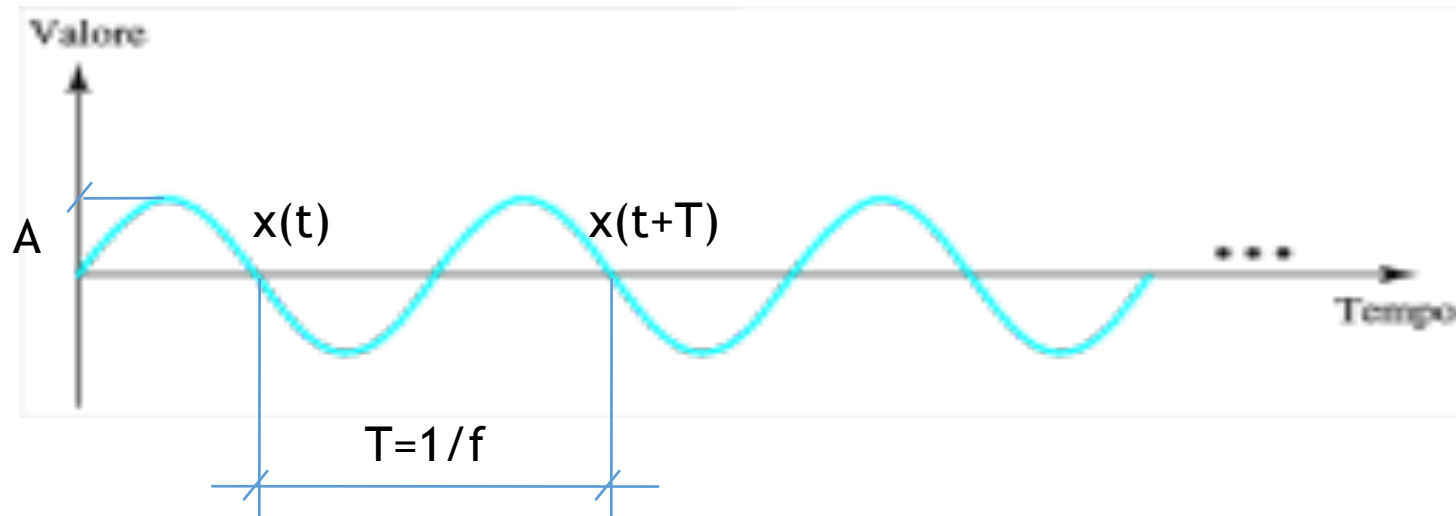
- Network performance is measured in two fundamental ways:
  - Bandwidth, but more properly **throughput**: Number of bits per second (bits/s or b/s) that can be transmitted over a communication link
  - **Delay, or latency**: the time between sending a message, and its reception
- 1 Kbps:  $1 \times 10^3$  bits/second =  $\sim 1 \times 2^{10}$  bits/sec
- 1 Mbps:  $1 \times 10^6$  bits/second =  $\sim 1 \times 2^{20}$  bits/sec
- 1 Gbps:  $1 \text{ Kbps} \cdot 1 \text{ Mbps} = \sim 1 \times 2^{30}$  bits/sec
- $1 \times 10^{-6}$  seconds to transmit each bit or imagine that a timeline, now each bit occupies  $1 \mu\text{s}$  (micro second) space.
- On a 2 Mbps link the width is  $0.5 \mu\text{s}$ .
- Smaller the width more will be transmission per unit time

# Performance

- Notice that the term **bandwidth** is quite confusing
- When you talk to informatics, it means **throughput** (that is, the number of bits we can transmit in a seconds)
- But when you talk to engineers and physicists, it means the **width of the frequency band** used in the communication
  - This is the correct definition.
  - So we will try to use *throughput* for the number of bits transmitted in one second, and *bandwidth* for the real bandwidth.

# What is the frequency band?

- Basic, stationary (i.e., periodic) signals are **sinusoids**
- A sinusoid is defined by three values ( $A, f, \varphi$ ):
  - Amplitude  $A$  [some value unit, e.g. volt, pascal, etc]
  - Frequency  $f$  [Hertz =  $1/s$ ]
  - Phase  $\varphi$  [rad]



$$\text{Sinusoid } x(t) = A \sin(2\pi f t + \varphi)$$

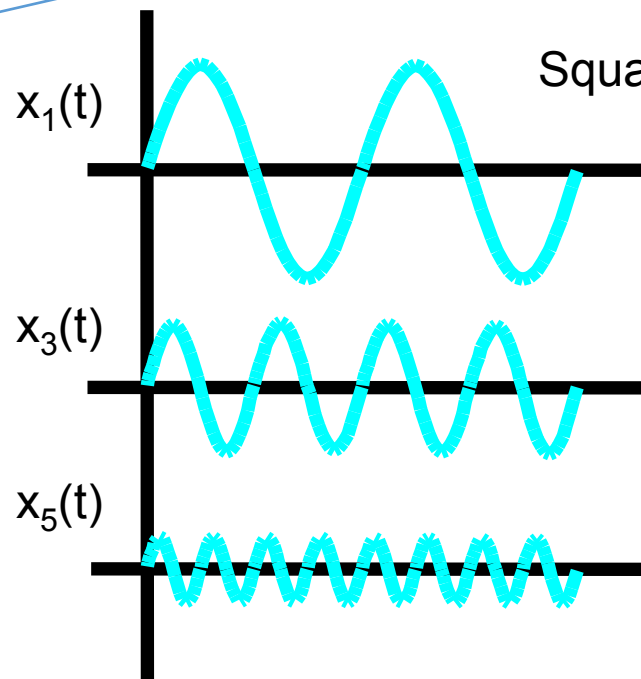
# What is the frequency spectrum?

Every "stationary" (=periodic) signal  $x(t)$  can be seen as a superposition of (phased) sinusoids:

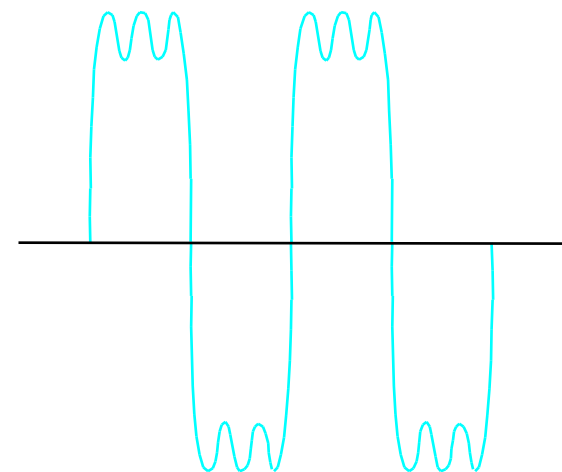
$$x(t) = \sum_i x_i(t) = \sum_i A_i \sin(2\pi f_i t + \varphi_i)$$



Jean Baptiste  
Joseph Fourier

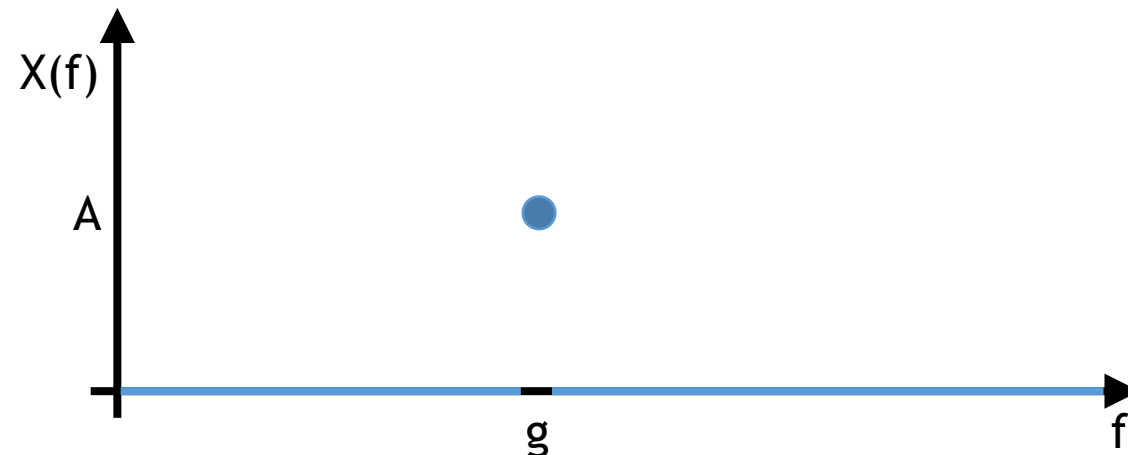


Square wave approximation  $x(t) = x_1(t) + x_3(t) + x_5(t)$



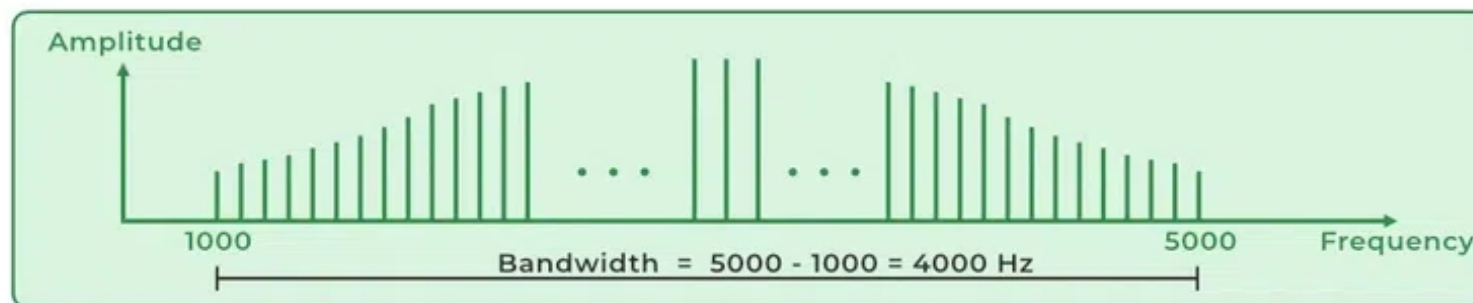
# Bandwidth as signal spectrum

- Let  $x(t)$  be a (stationary) signal
- For each frequency  $f$ , let us define  $X(f)$  as the amplitude (and the phase, using complex numbers) of the component of  $x$  of frequency  $f$
- $X$  is called the **spectrum** of the signal  $x$ , that is, the representation of  $x(t)$  in the space of frequencies
- Example: if  $x(t) = A \sin(2\pi g t + \varphi)$  (a single sinusoid), then  $X(g) = A$ , and  $X(f) = 0$  for  $f \neq g$ . Diagrammatically:

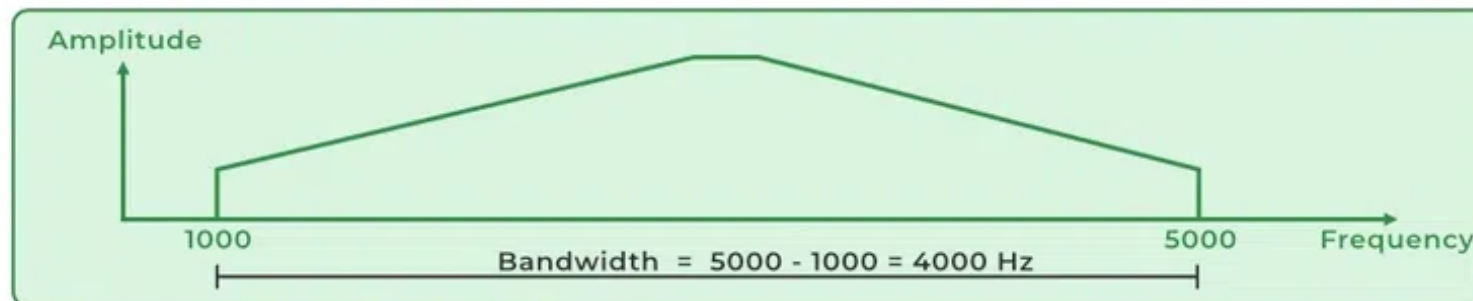


# Bandwidth as signal spectrum

- For a signal composed by many sinusoids,  $X(f)$  will have non-zero values for each component.
- The difference between the Maximum and the minimum frequencies with non-zero amplitude is the **bandwidth**
- Can be generalized to non-periodic signals



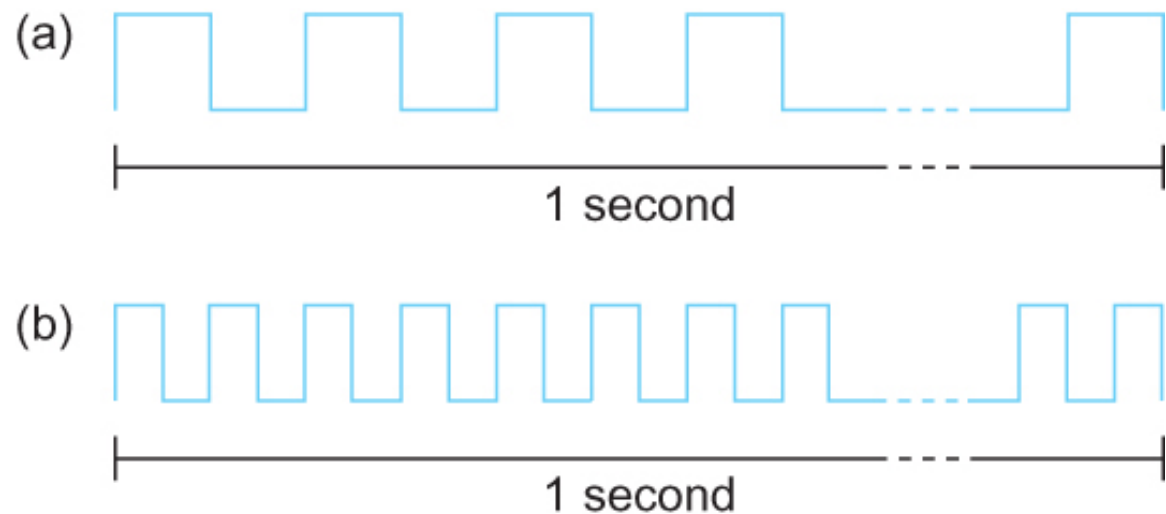
a. Bandwidth of a Periodic Signal



b. Bandwidth of a Non-Periodic Signal

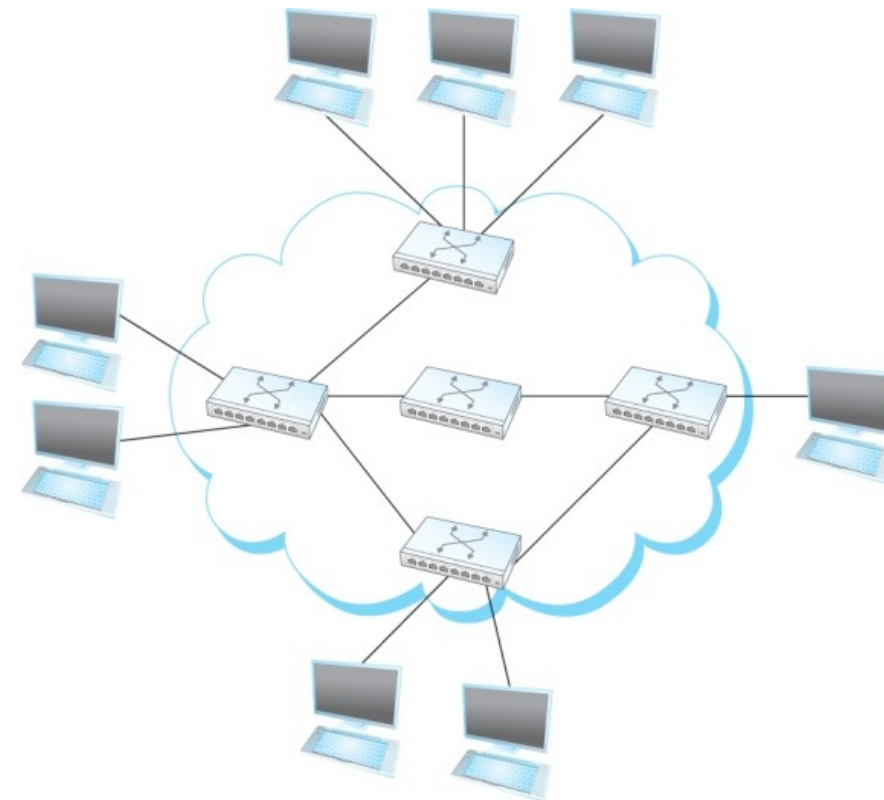
## Bandwidth as transmission speed

- Bits transmitted at a particular bandwidth can be regarded as having some width (i.e. temporal extension):
  - (a) bits transmitted at 1Mbps (each bit 1  $\mu$ s wide);
  - (b) bits transmitted at 2Mbps (each bit 0.5  $\mu$ s wide).



# Performance: Latency (or Delay)

- **Latency** = time taken for sending a message end-to-end
- **Round-trip-time (RTT)** = time taken for sending a message and receiving the answer, end-to-end
  - (Wrongly called the “ping”)
- It is specific for each pair of hosts, NOT a property of the network
- it varies during time
- $RTT_{A \rightarrow B} = Latency_{A \rightarrow B} + Latency_{B \rightarrow A}$   
     $= 2 * Latency_{A \rightarrow B}$   
    if the connection is symmetric



## Performance: Latency (or Delay)

**Latency = Propagation Time + Transmit Time + Queue Time**

where

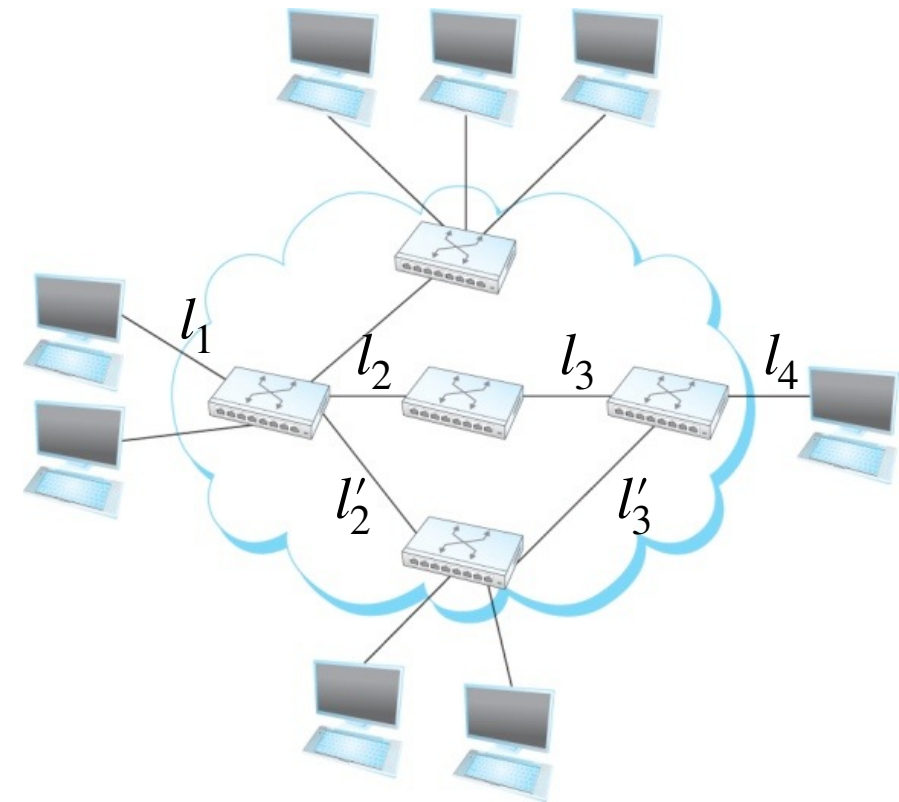
- **Propagation Time** = distance/speed of signal (in the specific mean)
  - Depends on physical laws, mostly constant once fixed the path
- **Transmit Time** = time spent transmitting the message = size of message/throughput
  - Depends on transmission speed and message size
- **Queue Time** = time spent inside computers, switches and routers
  - System and load dependent, may vary a lot in time

# Performance: Latency (or Delay)

- In a multi-hop network, this must be added for each link and switch crossed
- If the message crosses links  $l_1, \dots, l_n$ , each with latency  $L_1, \dots, L_n$ , the overall latency is

$$L = \sum_{i=1}^n L_i$$

- This means that
  - Latency contains also the delays introduced by intermediate devices
  - Fixed a path, the propagation and transmission times are basically constant, but the overall queue time depends (also) on the load of intermediate devices (which we may not be aware of)
  - If the path changes, also the latency changes



## Performance: Latency (or Delay)

**Latency = Propagation Time + Transmit Time + Queue Time**

- Small messages (e.g. few bits) => propagation is important
- Large messages => transmit time (i.e. throughput) is important
- **Velocity Factor:** how much the medium is fast w.r.t. the light in the vacuum (which is the highest speed possible)

$$VF_{\text{medium}} = \frac{\text{Speed of light in medium}}{\text{Speed of light in vacuum}}$$

- The lower the VF, the slower the signal propagates, the greater the delay introduced by signal propagation

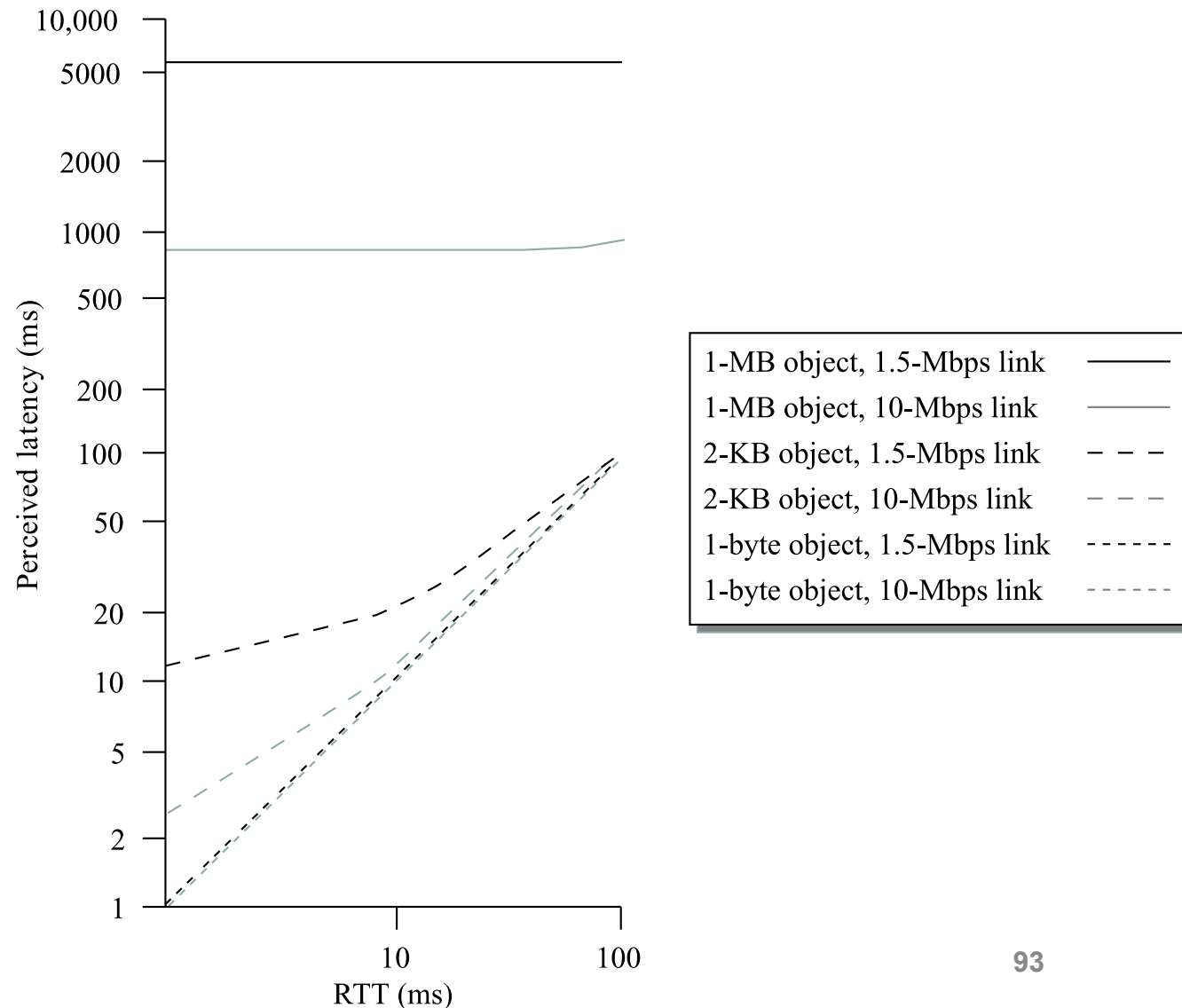
## Performance: Latency (or Delay)

- $$VF_{\text{medium}} = \frac{\text{Speed of light in medium}}{\text{Speed of light in vacuum}}$$
- The lower the VF, the slower the signal propagates, the greater the delay introduced by signal propagation
- E.g. speed of electromagnetic signal is  $3.0 \times 10^8$  m/s in a vacuum,  $2.3 \times 10^8$  m/s in a coaxial cable, and  $2.0 \times 10^8$  m/s in a fiber
- ...but optical fibers have much larger throughput and hence lower transmit time

| VF (%) | Cable                              |
|--------|------------------------------------|
| 74-79  | <a href="#">Cat-7 twisted pair</a> |
| 77     | RG-8/U                             |
| 67     | <a href="#">Optical fiber</a>      |
| 65     | RG-58A/U                           |
| 65     | Cat-6A twisted pair                |
| 64     | Cat-5e twisted pair                |
| 58.5   | Cat-3 twisted pair                 |

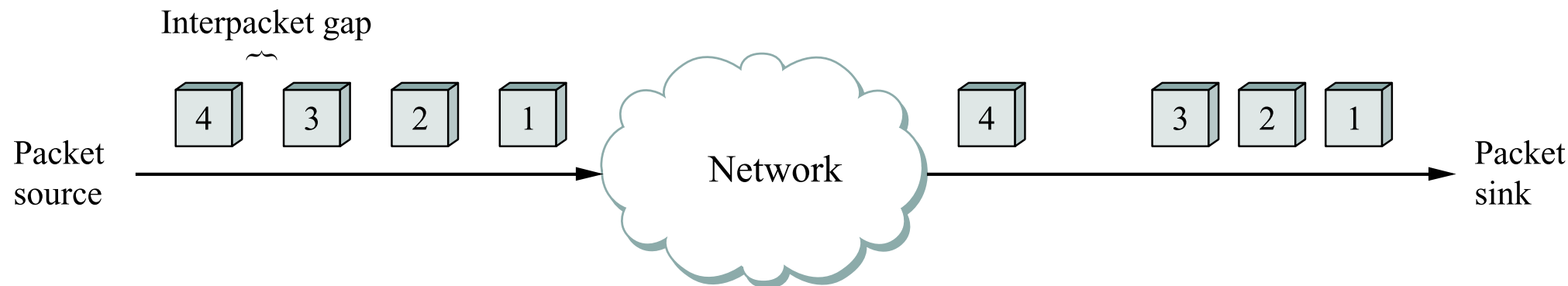
# Delay vs throughput

- Relative importance of throughput and delay of channels depends on application
  - For large file transfer, throughput is critical
  - For small messages (HTTP, NFS, SMS, etc.), delay is critical
- Unfortunately, it is difficult to optimize both at the same time
  - For having large throughput, we need to keep the channels full, but then the queues get longer, and hence the delay increases
  - For having low delays, queues should be empty or very short → low throughput

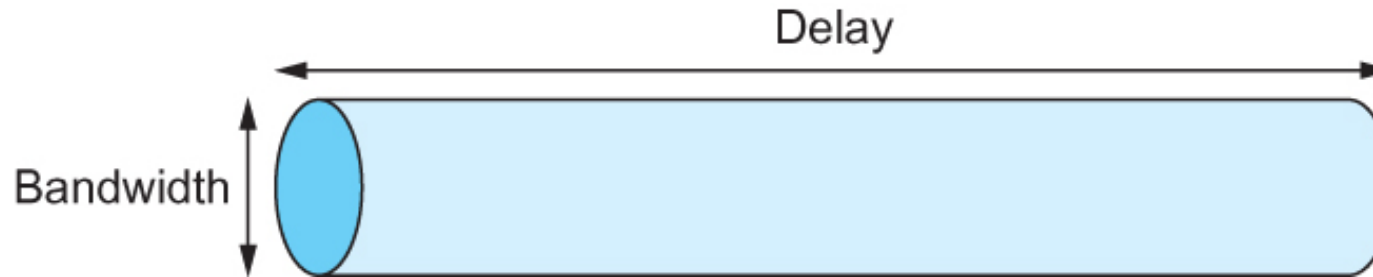


# Jitter

- Sometimes, delay is not a problem, but its variance is
  - E.g. multimedia, streaming audio/video
- Delay variance is called *jitter*
- Such variation is not usually introduced in links (they are passive, physical media) but rather by variable queues in switches (including routers), in multi-hop network



# Delay x Throughput



- We think the channel between a pair of processes as a hollow pipe
- Delay: length of the pipe (assimilable to propagation time)
- Throughput: width of the pipe
- Delay of 50 ms and throughput of 45 Mbps:  
$$50 \times 10^{-3} \text{ s} \times 45 \times 10^6 \text{ bits/s} = 2.25 \times 10^6 \text{ bits} = 280 \text{ KB data}$$
- This is the data which can be contained in the “pipe”, in flight

## Delay x Throughput

- The product Delay x Throughput represents how many bits the sender must transmit before the first bit arrives at the receiver, if the sender wants to keep the pipe full
- Takes another one-way delay to receive a response from the receiver
  - The time to go and come back is called **Round Trip Time (RTT)** - usually equal to  $2 \times \text{Delay}$  (if the channel is symmetric)
- If the sender does not “fill the pipe” (i.e., send a whole  $\text{RTT} \times \text{throughput}$  product’s worth of data before it stops to wait for a signal) it will not fully utilize the network (think of “bubbles” in the pipe)

## Delay x Throughput

- Some example Delay x Throughput (here the delay is RTT)
- (Here Bandwidth means Throughput - in bps)

| Link Type           | Bandwidth<br>(Typical) | Distance<br>(Typical) | Round-trip Delay | Delay × BW |
|---------------------|------------------------|-----------------------|------------------|------------|
| Dial-up             | 56 Kbps                | 10 km                 | 87 $\mu$ s       | 5 bits     |
| Wireless LAN        | 54 Mbps                | 50 m                  | 0.33 $\mu$ s     | 18 bits    |
| Satellite           | 45 Mbps                | 35,000 km             | 230 ms           | 10 Mb      |
| Cross-country fiber | 10 Gbps                | 4,000 km              | 40 ms            | 400 Mb     |

# Chapter 1 - Summary

- We have identified what we expect from a computer network
- We have defined a layered architecture for computer network that will serve as a blueprint for our design
- We have discussed the socket interface which will be used by applications for invoking the services of the network subsystem
- We have discussed some metrics which we can use to analyse the performance of computer networks